Generate Real Estate Homepage Code

https://gemini.google.com/app/0f563f1e71379c0c

User prompt: generate code for homepage of a website meant to help people find a better home

Response: Here is a clean, modern, and responsive homepage for a real estate website called **"NestFinder."**

It features a striking hero section with a search bar, a "Features" section to showcase the app's unique selling points, a "Featured Listings" grid, and a testimonial slider.

You can save this code as an `index.html` file and open it directly in your browser.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>NestFinder | Find Your Better Home</title>
    <!-- Google Fonts & FontAwesome Icons -->
    <link href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700&display=swap" rel="styleshee
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css">

    <style>
        /* --- CSS Reset & Variables --- */
        :root {
            --primary: #2563eb;
            --primary-hover: #1d4ed8;
            --secondary: #0f172a;
            --background: #f8fafc;
            --card-bg: #ffffff;
            --text-main: #334155;
            --text-muted: #64748b;
            --border: #e2e8f0;
            --radius: 12px;
            --transition: all 0.3s ease;
        }

        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
            font-family: 'Inter', sans-serif;
        }

        body {
            background-color: var(--background);
            color: var(--text-main);
            line-height: 1.6;
        }

        a {
            text-decoration: none;
            color: inherit;
        }

        /* --- Navigation --- */
        header {
            background: rgba(255, 255, 255, 0.95);
            backdrop-filter: blur(10px);
            position: fixed;
            width: 100%;
            top: 0;
            z-index: 1000;
            border-bottom: 1px solid var(--border);
        }

        .nav-container {
            max-width: 1200px;
            margin: 0 auto;
            padding: 1.25rem 2rem;
            display: flex;
            justify-content: space-between;
            align-items: center;
        }

        .logo {
            font-size: 1.5rem;
            font-weight: 700;
            color: var(--secondary);
```

```
    display: flex;
    align-items: center;
    gap: 0.5rem;
}

.logo i {
    color: var(--primary);
}

.nav-links {
    display: flex;
    gap: 2rem;
    list-style: none;
    align-items: center;
}

.nav-links a {
    font-weight: 500;
    color: var(--text-main);
    transition: var(--transition);
}

.nav-links a:hover {
    color: var(--primary);
}

.btn-primary {
    background: var(--primary);
    color: white !important;
    padding: 0.75rem 1.5rem;
    border-radius: var(--radius);
    font-weight: 600;
    transition: var(--transition);
}

.btn-primary:hover {
    background: var(--primary-hover);
    box-shadow: 0 4px 12px rgba(37, 99, 235, 0.2);
}

/* --- Hero Section --- */
.hero {
    padding: 10rem 2rem 6rem;
    background: linear-gradient(135deg, #f0fdf4 0%, #eff6ff 100%);
    text-align: center;
    position: relative;
    overflow: hidden;
}

.hero-content {
    max-width: 800px;
    margin: 0 auto;
}

.hero h1 {
    font-size: 3.5rem;
    color: var(--secondary);
    font-weight: 800;
    line-height: 1.2;
    margin-bottom: 1.5rem;
}

.hero h1 span {
    color: var(--primary);
}

.hero p {
    font-size: 1.25rem;
    color: var(--text-muted);
    margin-bottom: 3rem;
}

/* Search Bar Widget */
.search-container {
    background: var(--card-bg);
    padding: 1rem;
    border-radius: 16px;
    box-shadow: 0 10px 25px -5px rgba(0, 0, 0, 0.05), 0 8px 10px -6px rgba(0, 0, 0, 0.05);
    display: grid;
```

```
    grid-template-columns: repeat(4, 1fr) auto;
    gap: 1rem;
    align-items: center;
    max-width: 900px;
    margin: 0 auto;
}

.search-group {
    display: flex;
    flex-direction: column;
    text-align: left;
    padding: 0 0.5rem;
    border-right: 1px solid var(--border);
}

.search-group:nth-child(4) {
    border-right: none;
}

.search-group label {
    font-size: 0.75rem;
    text-transform: uppercase;
    font-weight: 700;
    color: var(--text-muted);
    margin-bottom: 0.25rem;
}

.search-group select, .search-group input {
    border: none;
    outline: none;
    font-size: 1rem;
    color: var(--secondary);
    font-weight: 500;
    background: transparent;
    width: 100%;
}

.search-btn {
    background: var(--primary);
    color: white;
    border: none;
    padding: 1rem 2rem;
    border-radius: var(--radius);
    font-size: 1rem;
    font-weight: 600;
    cursor: pointer;
    transition: var(--transition);
    display: flex;
    align-items: center;
    gap: 0.5rem;
}

.search-btn:hover {
    background: var(--primary-hover);
}

/* --- Core Features Section --- */
.section {
    padding: 6rem 2rem;
    max-width: 1200px;
    margin: 0 auto;
}

.section-header {
    text-align: center;
    margin-bottom: 4rem;
}

.section-header h2 {
    font-size: 2.25rem;
    color: var(--secondary);
    margin-bottom: 1rem;
}

.section-header p {
    color: var(--text-muted);
    font-size: 1.1rem;
}
```

```
.features-grid {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 2rem;
}

.feature-card {
    background: var(--card-bg);
    padding: 2.5rem;
    border-radius: var(--radius);
    border: 1px solid var(--border);
    transition: var(--transition);
}

.feature-card:hover {
    transform: translateY(-5px);
    box-shadow: 0 10px 20px rgba(0,0,0,0.03);
}

.feature-icon {
    width: 50px;
    height: 50px;
    background: #eff6ff;
    color: var(--primary);
    border-radius: 10px;
    display: flex;
    align-items: center;
    justify-content: center;
    font-size: 1.5rem;
    margin-bottom: 1.5rem;
}

.feature-card h3 {
    margin-bottom: 1rem;
    color: var(--secondary);
}

/* --- Featured Listings --- */
.listings-grid {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 2rem;
}

.listing-card {
    background: var(--card-bg);
    border-radius: var(--radius);
    border: 1px solid var(--border);
    overflow: hidden;
    transition: var(--transition);
}

.listing-card:hover {
    transform: translateY(-5px);
    box-shadow: 0 12px 24px rgba(0,0,0,0.06);
}

.listing-img {
    height: 220px;
    background-size: cover;
    background-position: center;
    position: relative;
}

/* Placeholder background images using Unsplash */
.img-1 { background-image: url('https://images.unsplash.com/photo-1570129477492-45c003edd2be?auto=format&fit
.img-2 { background-image: url('https://images.unsplash.com/photo-1564013799919-ab600027ffc6?auto=format&fit
.img-3 { background-image: url('https://images.unsplash.com/photo-1580587771525-78b9dba3b914?auto=format&fit

.tag {
    position: absolute;
    top: 1rem;
    left: 1rem;
    background: rgba(15, 23, 42, 0.85);
    backdrop-filter: blur(4px);
    color: white;
    padding: 0.25rem 0.75rem;
    border-radius: 20px;
    font-size: 0.8rem;
```

```
    font-weight: 600;
}

.listing-content {
    padding: 1.5rem;
}

.price {
    font-size: 1.5rem;
    font-weight: 700;
    color: var(--primary);
    margin-bottom: 0.5rem;
}

.listing-title {
    font-size: 1.2rem;
    color: var(--secondary);
    margin-bottom: 0.5rem;
}

.location {
    color: var(--text-muted);
    font-size: 0.9rem;
    margin-bottom: 1rem;
    display: flex;
    align-items: center;
    gap: 0.25rem;
}

.details {
    display: flex;
    justify-content: space-between;
    border-top: 1px solid var(--border);
    padding-top: 1rem;
    color: var(--text-muted);
    font-size: 0.9rem;
}

.details span i {
    margin-right: 0.25rem;
    color: var(--secondary);
}

/* --- Footer --- */
footer {
    background: var(--secondary);
    color: #94a3b8;
    padding: 4rem 2rem 2rem;
}

.footer-container {
    max-width: 1200px;
    margin: 0 auto;
    display: grid;
    grid-template-columns: 2fr repeat(3, 1fr);
    gap: 4rem;
    border-bottom: 1px solid #334155;
    padding-bottom: 3rem;
}

.footer-info .logo {
    color: white;
    margin-bottom: 1rem;
}

.footer-links h4 {
    color: white;
    margin-bottom: 1.25rem;
}

.footer-links ul {
    list-style: none;
}

.footer-links li {
    margin-bottom: 0.75rem;
}

.footer-links a:hover {
```

```
            color: white;
        }

        .copyright {
            text-align: center;
            max-width: 1200px;
            margin: 2rem auto 0;
            font-size: 0.9rem;
        }

        /* --- Responsive Design --- */
        @media (max-width: 968px) {
            .search-container {
                grid-template-columns: 1fr 1fr;
            }
            .search-group:nth-child(2) {
                border-right: none;
            }
            .search-btn {
                grid-column: span 2;
                justify-content: center;
            }
            .features-grid, .listings-grid, .footer-container {
                grid-template-columns: 1fr;
            }
            .hero h1 {
                font-size: 2.5rem;
            }
        }
    </style>
</head>
<body>

    <!-- Navigation Header -->
    <header>
        <nav class="nav-container">
            <a href="#" class="logo">
                <i class="fa-solid fa-house-chimney-window"></i> NestFinder
            </a>
            <ul class="nav-links">
                <li><a href="#features">Why Us</a></li>
                <li><a href="#listings">Explore</a></li>
                <li><a href="#">Neighborhoods</a></li>
                <li><a href="#" class="btn-primary">Sign In</a></li>
            </ul>
        </nav>
    </header>

    <!-- Hero Section -->
    <section class="hero">
        <div class="hero-content">
            <h1>Find a <span>Better Home</span> That Fits Your Life</h1>
            <p>We look beyond the floor plans to match you with a neighborhood, a community, and a space where you w

            <!-- Dynamic Search Widget -->
            <div class="search-container">
                <div class="search-group">
                    <label>Location</label>
                    <input type="text" placeholder="City, Zip, or Neighborhood" value="Austin, TX">
                </div>
                <div class="search-group">
                    <label>Home Type</label>
                    <select>
                        <option>Single Family</option>
                        <option>Townhouse</option>
                        <option>Apartment</option>
                        <option>Condo</option>
                    </select>
                </div>
                <div class="search-group">
                    <label>Price Range</label>
                    <select>
                        <option>$400k - $600k</option>
                        <option>$600k - $900k</option>
                        <option>$900k+</option>
                    </select>
                </div>
                <div class="search-group">
                    <label>Priority</label>
```

```
                <select>
                    <option>Top School District</option>
                    <option>Walkability</option>
                    <option>Quiet Suburb</option>
                    <option>Commute Friendly</option>
                </select>
            </div>
            <button class="search-btn">
                <i class="fa-solid fa-magnifying-glass"></i> Search
            </button>
        </div>
    </div>
</section>

<!-- Why NestFinder (Features) -->
<section class="section" id="features">
    <div class="section-header">
        <h2>What Makes a Home "Better"?</h2>
        <p>A great home is more than just square footage. It's about data-driven lifestyle matching.</p>
    </div>
    <div class="features-grid">
        <div class="feature-card">
            <div class="feature-icon"><i class="fa-solid fa-chart-line"></i></div>
            <h3>TrueCost™ Calculator</h3>
            <p>We calculate hidden costs like HOA fees, property taxes, and average utilities so you know your a
        </div>
        <div class="feature-card">
            <div class="feature-icon"><i class="fa-solid fa-smile"></i></div>
            <h3>Neighborhood Vibe Check</h3>
            <p>Get real insights into neighborhood noise levels, walkability, traffic safety, and community cult
        </div>
        <div class="feature-card">
            <div class="feature-icon"><i class="fa-solid fa-bolt"></i></div>
            <h3>Smart Match Alerts</h3>
            <p>Tell us your non-negotiables (e.g., backyard space, natural light, fast commute) and let our algo
        </div>
    </div>
</section>

<!-- Featured Listings -->
<section class="section" style="background-color: #ffffff; max-width: 100%; width: 100%; border-top: 1px solid v
    <div style="max-width: 1200px; margin: 0 auto;" id="listings">
        <div class="section-header">
            <h2>Highly Rated Matches Near You</h2>
            <p>Based on affordability, building quality, and neighborhood satisfaction scores.</p>
        </div>
        <div class="listings-grid">
            <!-- Card 1 -->
            <div class="listing-card">
                <div class="listing-img img-1">
                    <span class="tag">Top School District</span>
                </div>
                <div class="listing-content">
                    <div class="price">$545,000</div>
                    <h3 class="listing-title">The Willowcrest Estate</h3>
                    <div class="location"><i class="fa-solid fa-location-dot"></i> 742 Evergreen Terrace, Austin
                    <div class="details">
                        <span><i class="fa-solid fa-bed"></i> 4 Beds</span>
                        <span><i class="fa-solid fa-bath"></i> 3 Baths</span>
                        <span><i class="fa-solid fa-ruler-combined"></i> 2,400 sqft</span>
                    </div>
                </div>
            </div>
            <!-- Card 2 -->
            <div class="listing-card">
                <div class="listing-img img-2">
                    <span class="tag">Highly Walkable</span>
                </div>
                <div class="listing-content">
                    <div class="price">$620,000</div>
                    <h3 class="listing-title">Modern Urban Townhome</h3>
                    <div class="location"><i class="fa-solid fa-location-dot"></i> 112 Austin Blvd, Downtown</di
                    <div class="details">
                        <span><i class="fa-solid fa-bed"></i> 3 Beds</span>
                        <span><i class="fa-solid fa-bath"></i> 2.5 Baths</span>
                        <span><i class="fa-solid fa-ruler-combined"></i> 1,950 sqft</span>
                    </div>
                </div>
            </div>
```

```
                <!-- Card 3 -->
                <div class="listing-card">
                    <div class="listing-img img-3">
                        <span class="tag">Quiet Neighborhood</span>
                    </div>
                    <div class="listing-content">
                        <div class="price">$490,000</div>
                        <h3 class="listing-title">Sunlit Mid-Century Modern</h3>
                        <div class="location"><i class="fa-solid fa-location-dot"></i> 89 Pinecrest Way, West Lake</
                        <div class="details">
                            <span><i class="fa-solid fa-bed"></i> 3 Beds</span>
                            <span><i class="fa-solid fa-bath"></i> 2 Baths</span>
                            <span><i class="fa-solid fa-ruler-combined"></i> 1,700 sqft</span>
                        </div>
                    </div>
                </div>
            </div>
        </div>
    </section>

    <!-- Footer -->
    <footer>
        <div class="footer-container">
            <div class="footer-info">
                <a href="#" class="logo">
                    <i class="fa-solid fa-house-chimney-window"></i> NestFinder
                </a>
                <p>Helping you transition seamlessly into a place where you belong. Better homes for better lives.</
            </div>
            <div class="footer-links">
                <h4>Search</h4>
                <ul>
                    <li><a href="#">Buy a Home</a></li>
                    <li><a href="#">Rent an Apartment</a></li>
                    <li><a href="#">Find an Agent</a></li>
                </ul>
            </div>
            <div class="footer-links">
                <h4>Resources</h4>
                <ul>
                    <li><a href="#">Mortgage Guide</a></li>
                    <li><a href="#">Neighborhood Analytics</a></li>
                    <li><a href="#">Moving Checklist</a></li>
                </ul>
            </div>
            <div class="footer-links">
                <h4>Company</h4>
                <ul>
                    <li><a href="#">About Us</a></li>
                    <li><a href="#">Careers</a></li>
                    <li><a href="#">Contact Support</a></li>
                </ul>
            </div>
        </div>
        <div class="copyright">
            &copy; 2026 NestFinder Inc. All rights reserved. Built for a better tomorrow.
        </div>
    </footer>

</body>
</html>
```

Highlights of this design:

- **Lifestyle Search:** The hero search bar doesn't just ask for price and type; it includes a **"Priority"** filter (e.g., Top School District, Walkability) to emphasize the goal of finding a *better* home.
- **Value Propositions:** The features break down modern pain-points like transparent pricing (*TrueCost™*) and community atmosphere (*Vibe Check*).
- **Modern UI:** Clean card shadows, elegant spacing, subtle linear-gradients, and custom tags make the interface feel premium and trustworthy.

User prompt: Generate a login page for the website

Response: Here is a sleek, modern login page designed to seamlessly match the branding, colors, and typography of the **NestFinder** homepage.

It includes a clean split-screen layout (popular in modern SaaS and real estate platforms), featuring a beautiful background image on one side and a distraction-free, accessible login form on the other.

You can save this code as `login.html` and link it to your homepage.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Sign In | NestFinder</title>
    <link href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700&display=swap" rel="styleshee
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css">

    <style>
        /* --- CSS Variables (Matching Homepage) --- */
        :root {
            --primary: #2563eb;
            --primary-hover: #1d4ed8;
            --secondary: #0f172a;
            --background: #f8fafc;
            --card-bg: #ffffff;
            --text-main: #334155;
            --text-muted: #64748b;
            --border: #e2e8f0;
            --radius: 12px;
            --transition: all 0.3s ease;
        }

        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
            font-family: 'Inter', sans-serif;
        }

        body {
            background-color: var(--background);
            color: var(--text-main);
            height: 100vh;
            display: flex;
            overflow: hidden;
        }

        /* --- Split Layout --- */
        .login-container {
            display: flex;
            width: 100%;
            height: 100vh;
        }

        /* Left Side: Form Canvas */
        .form-side {
            flex: 1;
            display: flex;
            flex-direction: column;
            justify-content: center;
            padding: 3rem;
            background: var(--card-bg);
            max-width: 550px;
            position: relative;
        }

        /* Right Side: Visual Canvas */
        .image-side {
            flex: 1.5;
            background: linear-gradient(135deg, rgba(15, 23, 42, 0.7) 0%, rgba(37, 99, 235, 0.5) 100%),
                        url('https://images.unsplash.com/photo-1600585154340-be6161a56a0c?auto=format&fit=crop&w=120
            background-size: cover;
            background-position: center;
            display: flex;
            align-items: center;
            justify-content: center;
            padding: 4rem;
            color: white;
        }

        .image-content {
            max-width: 500px;
        }
```

```
.image-content h2 {
    font-size: 2.5rem;
    font-weight: 700;
    margin-bottom: 1rem;
    line-height: 1.2;
}

.image-content p {
    font-size: 1.1rem;
    color: #e2e8f0;
    line-height: 1.6;
}

/* --- Top Logo / Back Navigation --- */
.top-nav {
    position: absolute;
    top: 2rem;
    left: 3rem;
    right: 3rem;
    display: flex;
    justify-content: space-between;
    align-items: center;
}

.logo {
    font-size: 1.35rem;
    font-weight: 700;
    color: var(--secondary);
    text-decoration: none;
    display: flex;
    align-items: center;
    gap: 0.5rem;
}

.logo i {
    color: var(--primary);
}

.back-link {
    font-size: 0.9rem;
    font-weight: 500;
    color: var(--text-muted);
    text-decoration: none;
    display: flex;
    align-items: center;
    gap: 0.4rem;
    transition: var(--transition);
}

.back-link:hover {
    color: var(--primary);
}

/* --- Form Elements --- */
.form-wrapper {
    width: 100%;
    max-width: 400px;
    margin: 0 auto;
}

.form-header {
    margin-bottom: 2rem;
}

.form-header h1 {
    font-size: 2rem;
    color: var(--secondary);
    font-weight: 700;
    margin-bottom: 0.5rem;
}

.form-header p {
    color: var(--text-muted);
    font-size: 0.95rem;
}

.form-header a {
    color: var(--primary);
```

```
        text-decoration: none;
        font-weight: 600;
    }

    .form-header a:hover {
        text-decoration: underline;
    }

    /* Social Login Buttons */
    .social-buttons {
        display: grid;
        grid-template-columns: 1fr 1fr;
        gap: 1rem;
        margin-bottom: 1.5rem;
    }

    .btn-social {
        display: flex;
        align-items: center;
        justify-content: center;
        gap: 0.5rem;
        padding: 0.75rem;
        border: 1px solid var(--border);
        border-radius: var(--radius);
        background: white;
        font-size: 0.9rem;
        font-weight: 500;
        color: var(--secondary);
        cursor: pointer;
        transition: var(--transition);
    }

    .btn-social:hover {
        background: var(--background);
        border-color: #cbd5e1;
    }

    .divider {
        display: flex;
        align-items: center;
        text-align: center;
        margin: 1.5rem 0;
        color: var(--text-muted);
        font-size: 0.85rem;
    }

    .divider::before, .divider::after {
        content: '';
        flex: 1;
        border-bottom: 1px solid var(--border);
    }

    .divider:not(:empty)::before { margin-right: .75rem; }
    .divider:not(:empty)::after { margin-left: .75rem; }

    /* Inputs */
    .input-group {
        margin-bottom: 1.25rem;
    }

    .input-group label {
        display: block;
        font-size: 0.870rem;
        font-weight: 600;
        color: var(--secondary);
        margin-bottom: 0.5rem;
    }

    .input-wrapper {
        position: relative;
        display: flex;
        align-items: center;
    }

    .input-wrapper i {
        position: absolute;
        left: 1rem;
        color: var(--text-muted);
    }
```

```
.input-wrapper input {
    width: 100%;
    padding: 0.85rem 1rem 0.85rem 2.5rem;
    border: 1px solid var(--border);
    border-radius: var(--radius);
    font-size: 0.95rem;
    outline: none;
    transition: var(--transition);
}

.input-wrapper input:focus {
    border-color: var(--primary);
    box-shadow: 0 0 0 4px rgba(37, 99, 235, 0.1);
}

/* Form Utilities (Remember/Forgot) */
.form-utils {
    display: flex;
    justify-content: space-between;
    align-items: center;
    font-size: 0.875rem;
    margin-bottom: 1.5rem;
}

.remember-me {
    display: flex;
    align-items: center;
    gap: 0.5rem;
    cursor: pointer;
    color: var(--text-muted);
}

.remember-me input {
    accent-color: var(--primary);
    width: 16px;
    height: 16px;
}

.forgot-pass {
    color: var(--primary);
    text-decoration: none;
    font-weight: 500;
}

.forgot-pass:hover {
    text-decoration: underline;
}

/* Main Submit Button */
.btn-submit {
    width: 100%;
    background: var(--primary);
    color: white;
    border: none;
    padding: 0.85rem;
    border-radius: var(--radius);
    font-size: 1rem;
    font-weight: 600;
    cursor: pointer;
    transition: var(--transition);
}

.btn-submit:hover {
    background: var(--primary-hover);
    box-shadow: 0 4px 12px rgba(37, 99, 235, 0.2);
}

/* --- Responsive Design --- */
@media (max-width: 900px) {
    .image-side {
        display: none;
    }
    .form-side {
        max-width: 100%;
        padding: 2rem;
    }
    .top-nav {
        left: 2rem;
```

```
                right: 2rem;
            }
        }
    </style>
</head>
<body>

    <div class="login-container">

        <main class="form-side">
            <div class="top-nav">
                <a href="index.html" class="logo">
                    <i class="fa-solid fa-house-chimney-window"></i> NestFinder
                </a>
                <a href="index.html" class="back-link">
                    <i class="fa-solid fa-arrow-left"></i> Home
                </a>
            </div>

            <div class="form-wrapper">
                <div class="form-header">
                    <h1>Welcome Back</h1>
                    <p>Don't have an account? <a href="#">Create one free</a></p>
                </div>

                <div class="social-buttons">
                    <button class="btn-social">
                        <img src="https://authjs.dev/img/providers/google.svg" width="18" alt="Google"> Google
                    </button>
                    <button class="btn-social">
                        <img src="https://authjs.dev/img/providers/apple.svg" width="18" alt="Apple"> Apple
                    </button>
                </div>

                <div class="divider">or sign in with email</div>

                <form action="#" method="POST" onsubmit="event.preventDefault(); alert('Login simulated!');">
                    <div class="input-group">
                        <label for="email">Email Address</label>
                        <div class="input-wrapper">
                            <i class="fa-regular fa-envelope"></i>
                            <input type="email" id="email" placeholder="name@example.com" required autocomplete="ema
                        </div>
                    </div>

                    <div class="input-group">
                        <label for="password">Password</label>
                        <div class="input-wrapper">
                            <i class="fa-solid fa-lock"></i>
                            <input type="password" id="password" placeholder="••••••••" required autocomplete="curre
                        </div>
                    </div>

                    <div class="form-utils">
                        <label class="remember-me">
                            <input type="checkbox" id="remember">
                            <span>Keep me signed in</span>
                        </label>
                        <a href="#" class="forgot-pass">Forgot password?</a>
                    </div>

                    <button type="submit" class="btn-submit">Sign In to NestFinder</button>
                </form>
            </div>
        </main>

        <section class="image-side">
            <div class="image-content">
                <h2>Save houses. Compare scores. Find your fit.</h2>
                <p>Log in to access your curated dashboard, saved neighborhood vibe-checks, and custom TrueCost™ cal
            </div>
        </section>

    </div>

</body>
</html>
```

Key Features of the Login Page:

- **Cohesive Branding:** Utilizes the same modern color palette (`#2563eb` Inter blue and deep `#0f172a` slate) to ensure users feel like they never left the main app experience.
- **Social Sign-In Integration:** Ready-styled layout spaces for Google and Apple quick-login wrappers.
- **Fully Responsive:** On mobile devices or smaller viewports, the resource-heavy image panel gracefully slides away (`display: none`), putting 100% focus on a clean, thumb-friendly form.
- **UX Accessibility Details:** Pre-configured input wrappers utilizing icon indicators, standard placeholder states, and strict form semantics like native browser `autocomplete` suggestions.

User prompt: The website is meant to adhere to these instructions. Provide any necessary changes to the existing codes and create every necessary addition. Explain the logic and adherence to the instructions of each seperately

Response: This is a fantastic visual foundation for the NestFinder project! The design is clean, modern, and fits the real estate theme perfectly. However, looking at the guidelines in the "Full Stack - Adv. Course - Final Project.pdf", we need to make a significant structural pivot.

The current files are static HTML and CSS. The project requirements mandate a complete end-to-end Full Stack application. To adhere to the instructions, we must transition this static code into a **React Single Page Application (SPA)** for the frontend and build a **Node.js/Express** backend with a **MongoDB** database.

Here is a breakdown of the necessary changes, the logic behind them, and how they adhere strictly to your assignment document.

1. The Frontend: Moving to React (SPA)

The Requirement: The client side must be a Single Page Application built with React, utilizing components, Hooks (`useState`, `useEffect`), and Axios for API calls.

The Logic & Changes: We need to break down your static HTML into reusable React components (e.g., `<Navbar/>`, `<Hero/>`, `<LoginForm/>`). We also need to add validation and API integration.

Here is how your Login page should be adapted into a React component to meet the course requirements for **React Form Validation** and **Axios**:

```
// src/components/Login.jsx
import React, { useState } from 'react';
import axios from 'axios';
import { useAuthContext } from '../context/AuthContext'; // Required Context API

const Login = () => {
    const [email, setEmail] = useState('');
    const [password, setPassword] = useState('');
    const [error, setError] = useState('');
    const [isLoading, setIsLoading] = useState(false); // UX Loading State required
    const { loginUser } = useAuthContext();

    const handleLogin = async (e) => {
        e.preventDefault();

        // Basic React Form Validation
        if (!email || !password) {
            return setError('All fields are required');
        }

        setIsLoading(true);
        try {
            // Axios API call required by the syllabus
            const response = await axios.post('http://localhost:5000/api/auth/login', {
                email,
                password
            });

            loginUser(response.data.token);
            // Redirect logic here...
        } catch (err) {
            setError(err.response?.data?.message || 'Login failed');
        } finally {
            setIsLoading(false);
        }
    };

    return (
        <div className="login-container">
            {/* Using your existing CSS classes */}
            <form onSubmit={handleLogin}>
                {error && <div className="error-message">{error}</div>}

                <input
                    type="email"
```

```
                    value={email}
                    onChange={(e) => setEmail(e.target.value)}
                    placeholder="name@example.com"
                />
                <input
                    type="password"
                    value={password}
                    onChange={(e) => setPassword(e.target.value)}
                    placeholder="••••••••"
                />

                <button type="submit" disabled={isLoading}>
                    {isLoading ? 'Signing In...' : 'Sign In'}
                </button>
            </form>
        </div>
    );
};

export default Login;
```

- **Adherence Check:** This implements Hooks (`useState`), `Axios` for the POST request, front-end validation, and includes a loading state for UX, which is explicitly requested.

2. State Management: Redux & Context API

The Requirement: The project explicitly demands the use of *both* the Context API (for global state like user details) and REDUX (for complex application states like property listings).

The Logic & Changes:

- **Context API:** Use this to wrap your application to store the user's JWT token and authentication status.
- **Redux:** Create a Redux store with Actions and Reducers to fetch, store, and filter the "Featured Listings" from your database.

3. The Backend: Node.js, Express & Security

The Requirement: A secure REST API built with Node.js and Express, implementing a complete Login/Register flow, `bcrypt` for passwords, and `JWT` for protecting routes. Incoming data must be validated with `JOI`.

The Logic & Changes: You must create a completely separate backend folder. Here is the code for the backend Login route to adhere to the requirements:

```
// server/controllers/authController.js
const bcrypt = require('bcrypt'); // Required for encryption
const jwt = require('jsonwebtoken'); // Required for authentication
const Joi = require('joi'); // Required for server-side validation
const User = require('../models/User'); // Mongoose Model

// JOI Validation Schema
const loginSchema = Joi.object({
    email: Joi.string().email().required(),
    password: Joi.string().min(6).required()
});

exports.login = async (req, res, next) => {
    try {
        // Validation step
        const { error } = loginSchema.validate(req.body);
        if (error) return res.status(400).json({ message: error.details[0].message });

        const { email, password } = req.body;

        // Mongoose query
        const user = await User.findOne({ email });
        if (!user) return res.status(401).json({ message: 'Invalid credentials' });

        // Bcrypt comparison step
        const isMatch = await bcrypt.compare(password, user.password);
        if (!isMatch) return res.status(401).json({ message: 'Invalid credentials' });

        // JWT generation step
        const token = jwt.sign({ id: user._id }, process.env.JWT_SECRET, { expiresIn: '1h' });

        res.status(200).json({ token, user: { id: user._id, email: user.email } });
    } catch (err) {
        next(err); // Passing to global Error Handling middleware
    }
};
```

- **Adherence Check:** This implements server-side JOI validation, bcrypt password checking, JWT generation, and pushes errors to a global error handler, fulfilling all backend security requirements.

4. Database & Media Uploads

The Requirement: MongoDB mapped with Mongoose schemas, and utilizing `Multer` for file uploads.

The Logic & Changes: Instead of hardcoding the house images via Unsplash URLs in the CSS, you will need to allow users (like real estate agents) to upload images of houses.

- **Action:** You must set up a Mongoose Schema for `Listing` that includes fields like `title`, `price`, `location`, and `imagePath`.
- **Action:** Create a route in Express that uses the `Multer` middleware to handle `multipart/form-data` file uploads, saving the files so they can be dynamically rendered in React.

Summary of Next Steps

To get this ready for your presentation, you should organize your workspace into two distinct folders: `client` (React) and `server` (Node.js). Would you like me to help you generate the **Mongoose Schema** for the real estate listings or the **Redux slice** to manage the property search state next?

User prompt: Should I replace the existing login page with code number 1, add code number 1 to the existing login page, or create a new file for code number 1 while keeping the existing login page intact?

Response: I completely understand the confusion! Transitioning from static web design to a full-stack environment involves a big shift in how your files are structured.

To answer your question directly: You should **create a new file** (named `Login.jsx`) for Code Number 1. Ultimately, this React component will **replace** your existing static `login.html` file. However, you should keep the original HTML file handy for now so you can easily copy your CSS and layout over to the new setup.

Here is a straightforward breakdown of why this is necessary and how to execute it:

Why Replace the Static HTML?

Your project guidelines explicitly require the frontend to be a "Single Page Application" (SPA) built with React using components.

- In traditional web development, a user clicking "Sign In" requests a completely separate `login.html` file from the server.
- In a React SPA, the server only ever sends *one* HTML file.
- React handles the navigation by dynamically swapping out Javascript components (like the `Login.jsx` code I provided) to instantly change what the user sees without reloading the page.

How to Merge Your Design with React

You definitely do not want to lose the sleek UI we built earlier! Here is the standard workflow for migrating static HTML into a React app:

- **Step 1: Create the Component File:** Inside your React project folder (usually in `src/components`), create a new file named `Login.jsx` and paste the logic from Code Number 1 into it.
- **Step 2: Migrate the Styles:** Copy all the CSS from the `<style>` tags of your original `login.html` file. Paste it into a new stylesheet in your React project (e.g., `Login.css`), and import it at the top of your React component using `import './Login.css';`.
- **Step 3: Port the HTML Structure:** Take the visual HTML elements from your original static page and carefully place them inside the `return (...)` block of your new `Login.jsx` component.
- **Step 4: Convert HTML to JSX:** Because React uses JSX, you will need to update standard HTML attributes. Most notably, change every instance of `class=` to `className=`.

By following these steps, you maintain the exact visual design of the original login page while strictly adhering to the syllabus requirement to use a React component-based architecture.

Would you like me to do the heavy lifting and provide the complete, combined `Login.jsx` code that merges the beautiful UI from the first step with the functional React logic from the second step?

User prompt: You did not provide a code for part 2

Response: You are absolutely right, and I appreciate you keeping me on track! I explained the logic for Part 2 but completely skipped providing the actual code. Let's fix that right now.

The syllabus strictly requires the implementation of **both** the Context API for global state and Redux for complex application state, including actions, reducers, and a store.

Here is how you can set up both systems side-by-side to perfectly meet those requirements.

1. Context API (User Authentication)

The Context API is ideal for data that needs to be accessed globally without passing props down manually, such as tracking if a user is currently logged in.

Create a file named `AuthContext.jsx` in your `src/context` folder:

```
import React, { createContext, useContext, useState } from 'react';

// Create the Context
const AuthContext = createContext();

// Create the Provider Component
export const AuthProvider = ({ children }) => {
    const [userToken, setUserToken] = useState(localStorage.getItem('token') || null);

    const loginUser = (token) => {
        localStorage.setItem('token', token);
        setUserToken(token);
    };

    const logoutUser = () => {
        localStorage.removeItem('token');
        setUserToken(null);
    };

    return (
        <AuthContext.Provider value={{ userToken, loginUser, logoutUser }}>
            {children}
        </AuthContext.Provider>
    );
};

// Custom hook to use the Auth Context
export const useAuthContext = () => useContext(AuthContext);
```

2. Redux Toolkit (Property Listings)

Redux is required for the more complex parts of your application. We will use it to handle the asynchronous API calls for fetching your real estate listings via Axios.

First, create a slice file named `listingsSlice.js` in your `src/redux` folder:

```
import { createSlice, createAsyncThunk } from '@reduxjs/toolkit';
import axios from 'axios';

// Asynchronous action using Axios as required
export const fetchListings = createAsyncThunk('listings/fetch', async () => {
    const response = await axios.get('http://localhost:5000/api/listings');
    return response.data;
});

const listingsSlice = createSlice({
    name: 'listings',
    initialState: { items: [], status: 'idle', error: null },
    reducers: {},
    extraReducers: (builder) => {
        builder
            .addCase(fetchListings.pending, (state) => {
                state.status = 'loading';
            })
            .addCase(fetchListings.fulfilled, (state, action) => {
                state.status = 'succeeded';
                state.items = action.payload;
            })
            .addCase(fetchListings.rejected, (state, action) => {
                state.status = 'failed';
                state.error = action.error.message;
            });
    },
});

export default listingsSlice.reducer;
```

Next, create your Redux store in `store.js`:

```
import { configureStore } from '@reduxjs/toolkit';
import listingsReducer from './listingsSlice';

export const store = configureStore({
    reducer: {
        listings: listingsReducer
```

```
    }
});
```

By wrapping your main `App.js` with **both** the `<AuthProvider>` and the Redux `<Provider store="{store}">`, you successfully fulfill the advanced state management requirements of your final project.

Would you like me to provide the combined `Login.jsx` code we discussed earlier next, or should we move straight to setting up the backend Mongoose models?

User prompt: Do the 3 most recent codes adhere to the instructions? Explain how

Response: Yes, the three most recent codes adhere perfectly to the requirements outlined in your "Full Stack - Adv. Course - Final Project.pdf" document.

Here is a straightforward breakdown of how each code snippet satisfies the syllabus instructions:

1. `Login.jsx` (React Component)

This code was designed to hit multiple frontend requirements simultaneously:

- **React SPA & Hooks:** It is built as a functional component utilizing the `useState` Hook to manage local data, strictly following the frontend structural requirements.
- **API Communication:** It utilizes the `Axios` library to perform a POST request to your backend, fulfilling the requirement to handle all API calls via Axios.
- **Form Validation:** It includes a manual check (`if (!email || !password)`) before submitting, satisfying the requirement to implement React Form Validation.
- **UX/UI Guidelines:** It includes an `isLoading` state that disables the button and changes its text, meeting the specific UX requirement to provide a Loading State (indication during data loading).

2. `AuthContext.jsx` (Context API)

The syllabus specifically mandates the use of **both** Context API and Redux.

- **Global User State:** This file creates a global state using the Context API specifically to hold the connected user's token and authentication status, matching the exact use-case example provided in the syllabus ("for example, for the connected user details").

3. `listingsSlice.js` & `store.js` (Redux Toolkit)

This code handles the other half of the strict state management requirement.

- **Complex State Management:** It sets up REDUX to manage the property listings data, which represents the core, complex data of your application.
- **Store, Actions, and Reducers:** It configures a central `Store`, defines Actions (using `createAsyncThunk` for fetching data), and builds Reducers to handle the success, failure, and loading states, completely fulfilling the Redux structural requirements.

You are on a great path by separating these responsibilities!

Would you like me to provide the complete, combined `Login.jsx` code that merges the beautiful UI from the very first step with the functional React logic we just verified?

User prompt: What file type should the backend login code be?

Response: The backend login code should be a standard **JavaScript (`.js`)** file.

Based on the instructions in the "Full Stack - Adv. Course - Final Project.pdf":

- The backend must be built using **Node.js & Express**, which natively uses JavaScript.
- The syllabus also strictly requires clean code that is divided into logical folders, specifically mentioning **Controllers, Models, and Routes**.

Therefore, the backend login logic provided earlier should be saved as a JavaScript file, typically named something like `authController.js`, and placed inside a `controllers` folder within your server directory.

User prompt: This is the project at its current state. Are there any new folders or files that need to be made?

Response: Based on the provided "image_426923.png", it is great that you have started creating your files, but you definitely need a major structural reorganization.

According to the "Full Stack - Adv. Course - Final Project.pdf" document, your project is required to have clean code that is logically divided into specific folders. Right now, your frontend files (`login.jsx`, HTML files) are sitting in the root directory, and your backend folder is missing the required sub-architectures.

Here is exactly what you need to create and reorganize to meet the strict grading criteria:

1. Frontend Architecture (React SPA)

You must create a dedicated frontend folder (e.g., `client` or `frontend`) to hold your Single Page Application. Inside that folder, you should create:

- **`src/components/`**: Move your `login.jsx` file here. You will also eventually convert your `homepage.html` into a `Homepage.jsx` component here.
- **`src/context/`**: Create this to hold your `AuthContext.jsx` file for global user state.
- **`src/redux/`**: Create this to hold your `store.js` and `listingsSlice.js` for complex state management.

2. Backend Architecture (Node.js/Express)

Inside your existing `backend` folder, you must create the specific folders explicitly requested by the syllabus:

- **`controllers/`**: Move your `login.js` file here (and ideally rename it to `authController.js` for clarity).
- **`models/`**: Create this folder to hold your Mongoose schemas (e.g., `User.js`, `Listing.js`).
- **`routes/`**: Create this to define your Express API endpoints (e.g., `authRoutes.js`).
- **`middlewares/`**: Create this to hold your global error handling and JWT protection logic.
- **`server.js`** (or `app.js`): You need an entry point file in the root of the `backend` folder to initialize your Express server and connect to MongoDB via Mongoose.

3. Cleanup of Static Files

- The syllabus requires a React SPA. Therefore, `homepage.html` and `log in.html` should not remain in your final production build. Use them as references to build your React `.jsx` components, and then delete or move them to an archive folder so they don't clutter your repository.

Would you like me to guide you on how to initialize the standard React environment (like using Vite or Create React App) inside your new frontend folder?

User prompt: Write me the code for homepage.jsx

`Response:`

```
import React, { useEffect } from 'react';
import { useDispatch, useSelector } from 'react-redux';
import { fetchListings } from '../redux/listingsSlice';
import { useAuthContext } from '../context/AuthContext';
import './Homepage.css';

const Homepage = () => {
    const dispatch = useDispatch();
    const { userToken } = useAuthContext();
    const { items: listings, status, error } = useSelector((state) => state.listings);

    useEffect(() => {
        if (status === 'idle') {
            dispatch(fetchListings());
        }
    }, [status, dispatch]);

    return (
        <div className="homepage">
            <header>
                <nav className="nav-container">
                    <a href="#" className="logo">
                        <i className="fa-solid fa-house-chimney-window"></i> NestFinder
                    </a>
                    <ul className="nav-links">
                        <li><a href="#features">Why Us</a></li>
                        <li><a href="#listings">Explore</a></li>
                        <li>{userToken ? <a href="/dashboard" className="btn-primary">Dashboard</a> : <a href="/logi
                    </ul>
                </nav>
            </header>

            <section className="hero">
                <div className="hero-content">
                    <h1>Find a <span>Better Home</span> That Fits Your Life</h1>
                    <p>We look beyond the floor plans to match you with a neighborhood, a community, and a space whe

                    <div className="search-container">
                        <div className="search-group">
                            <label>Location</label>
                            <input type="text" placeholder="City, Zip, or Neighborhood" defaultValue="Austin, TX" />
```

```
                </div>
                <div className="search-group">
                    <label>Home Type</label>
                    <select defaultValue="Single Family">
                        <option>Single Family</option>
                        <option>Townhouse</option>
                        <option>Apartment</option>
                        <option>Condo</option>
                    </select>
                </div>
                <div className="search-group">
                    <label>Price Range</label>
                    <select defaultValue="$400k - $600k">
                        <option>$400k - $600k</option>
                        <option>$600k - $900k</option>
                        <option>$900k+</option>
                    </select>
                </div>
                <div className="search-group">
                    <label>Priority</label>
                    <select defaultValue="Top School District">
                        <option>Top School District</option>
                        <option>Walkability</option>
                        <option>Quiet Suburb</option>
                        <option>Commute Friendly</option>
                    </select>
                </div>
                <button className="search-btn">
                    <i className="fa-solid fa-magnifying-glass"></i> Search
                </button>
            </div>
        </div>
    </section>

    <section className="section" id="features">
        <div className="section-header">
            <h2>What Makes a Home "Better"?</h2>
            <p>A great home is more than just square footage. It's about data-driven lifestyle matching.</p>
        </div>
        <div className="features-grid">
            <div className="feature-card">
                <div className="feature-icon"><i className="fa-solid fa-chart-line"></i></div>
                <h3>TrueCost™ Calculator</h3>
                <p>We calculate hidden costs like HOA fees, property taxes, and average utilities so you kno
            </div>
            <div className="feature-card">
                <div className="feature-icon"><i className="fa-solid fa-smile"></i></div>
                <h3>Neighborhood Vibe Check</h3>
                <p>Get real insights into neighborhood noise levels, walkability, traffic safety, and commun
            </div>
            <div className="feature-card">
                <div className="feature-icon"><i className="fa-solid fa-bolt"></i></div>
                <h3>Smart Match Alerts</h3>
                <p>Tell us your non-negotiables and let our algorithm find the hidden gems.</p>
            </div>
        </div>
    </section>

    <section className="section listings-section" id="listings">
        <div className="section-header">
            <h2>Highly Rated Matches Near You</h2>
            <p>Based on affordability, building quality, and neighborhood satisfaction scores.</p>
        </div>

        {status === 'loading' && <p className="loading-state">Loading your property matches...</p>}
        {error && <p className="error-state">Failed to load listings: {error}</p>}

        <div className="listings-grid">
            {listings && listings.map((listing) => (
                <div className="listing-card" key={listing._id || listing.id}>
                    <div className="listing-img" style={{ backgroundImage: `url(${listing.image || 'https://
                        <span className="tag">{listing.tag || 'Featured Match'}</span>
                    </div>
                    <div className="listing-content">
                        <div className="price">${listing.price?.toLocaleString()}</div>
                        <h3 className="listing-title">{listing.title}</h3>
                        <div className="location"><i className="fa-solid fa-location-dot"></i> {listing.loca
                        <div className="details">
                            <span><i className="fa-solid fa-bed"></i> {listing.beds} Beds</span>
```

```
                                    <span><i className="fa-solid fa-bath"></i> {listing.baths} Baths</span>
                                    <span><i className="fa-solid fa-ruler-combined"></i> {listing.sqft} sqft</span>
                                </div>
                            </div>
                        </div>
                    ))}
                </div>
            </section>

            <footer>
                <div className="footer-container">
                    <div className="footer-info">
                        <a href="#" className="logo">
                            <i className="fa-solid fa-house-chimney-window"></i> NestFinder
                        </a>
                        <p>Helping you transition seamlessly into a place where you belong.</p>
                    </div>
                </div>
                <div className="copyright">
                    &copy; 2026 NestFinder Inc. All rights reserved.
                </div>
            </footer>
        </div>
    );
};

export default Homepage;
```

User prompt: Generate the code for authContext.jsx

Response:

```
import React, { createContext, useContext, useState, useEffect } from 'react';

const AuthContext = createContext();

export const AuthProvider = ({ children }) => {
    const [user, setUser] = useState(null);
    const [userToken, setUserToken] = useState(localStorage.getItem('token') || null);

    useEffect(() => {
        if (userToken) {
            localStorage.setItem('token', userToken);
        } else {
            localStorage.removeItem('token');
        }
    }, [userToken]);

    const loginUser = (token, userData) => {
        setUserToken(token);
        setUser(userData);
        localStorage.setItem('token', token);
    };

    const logoutUser = () => {
        setUserToken(null);
        setUser(null);
        localStorage.removeItem('token');
    };

    return (
        <AuthContext.Provider value={{ user, userToken, loginUser, logoutUser }}>
            {children}
        </AuthContext.Provider>
    );
};

export const useAuthContext = () => {
    const context = useContext(AuthContext);
    if (!context) {
        throw new Error('useAuthContext must be used within an AuthProvider');
    }
    return context;
};
```

User prompt: Make store.js

```
Response: import { configureStore } from '@reduxjs/toolkit';
import listingsReducer from './listingsSlice';

export const store = configureStore({
    reducer: {
        listings: listingsReducer
    }
});

export default store;
```

User prompt: Now listingsSlice.js

```
Response: import { createSlice, createAsyncThunk } from '@reduxjs/toolkit';
import axios from 'axios';

export const fetchListings = createAsyncThunk(
    'listings/fetchListings',
    async (_, { rejectWithValue }) => {
        try {
            const response = await axios.get('http://localhost:5000/api/listings');
            return response.data;
        } catch (error) {
            return rejectWithValue(error.response?.data?.message || error.message);
        }
    }
);

const listingsSlice = createSlice({
    name: 'listings',
    initialState: {
        items: [],
        status: 'idle',
        error: null
    },
    reducers: {},
    extraReducers: (builder) => {
        builder
            .addCase(fetchListings.pending, (state) => {
                state.status = 'loading';
                state.error = null;
            })
            .addCase(fetchListings.fulfilled, (state, action) => {
                state.status = 'succeeded';
                state.items = action.payload;
            })
            .addCase(fetchListings.rejected, (state, action) => {
                state.status = 'failed';
                state.error = action.payload;
            });
    }
});

export default listingsSlice.reducer;
```

User prompt: This is the way the project is organized now. is there anything i should add, other than the server file>

Response: Your overall folder structure looks much cleaner and aligns far better with full-stack requirements. To make the project completely functional and compliant with your project guidelines, you need several essential backend and frontend files beyond just the server file.

Frontend Additions (`/frontend`)

- **`src/App.jsx`**: The core application component that handles page routing (e.g., using `react-router-dom`) between your Homepage and Login pages.
- **`src/main.jsx` (or `index.js`)**: The entry point where you wrap your application with both the Redux `<Provider store="{store}">` and Context `<AuthProvider>`.
- **`components/Register.jsx`**: A registration component, as a full Register process is required alongside Login.
- **`components/ProtectedRoute.jsx`**: A wrapper component to protect client-side routes for authenticated users.
- **CSS Stylesheets**: Separate `.css` files (e.g., `Homepage.css`, `Login.css`) to styling your components.
- **`package.json`**: The node configuration file tracking frontend dependencies (`react`, `react-redux`, `@reduxjs/toolkit`, `axios`, `react-router-dom`).

Backend Additions (`/backend`)

- **`server.js`**: The main Express server entry point to initialize your app, connect to MongoDB via Mongoose, and listen on a port.
- **`models/User.js`** & **`models/Listing.js`**: Mongoose schemas defining user data and property listing data structures.
- **`controllers/authController.js`** & **`controllers/listingController.js`**: Controller logic handling request/response flows for authentication and property CRUD actions.
- **`routes/authRoutes.js`** & **`routes/listingRoutes.js`**: Express router files defining REST API endpoints.
- **`middlewares/authMiddleware.js`**: Middleware to verify JWT tokens and secure protected API endpoints.
- **`middlewares/errorMiddleware.js`**: The global error-handling middleware required by the syllabus.
- **`middlewares/uploadMiddleware.js`**: Multer configuration middleware to process file/image uploads.
- **`uploads/`**: A folder to store uploaded media files locally before serving them dynamically.
- **`.env`**: Environment file storing sensitive keys (e.g., `PORT`, `MONGO_URI`, `JWT_SECRET`).
- **`package.json`**: Tracking backend dependencies (`express`, `mongoose`, `bcrypt`, `jsonwebtoken`, `joi`, `multer`, `dotenv`, `cors`).

Root Cleanup

- Delete or archive `homepage.html` and `log in.html` since React components now replace them.

User prompt: Write the codes needed for each

Response: **backend/.env**

```
PORT=5000
MONGO_URI=mongodb://127.0.0.1:27017/nestfinder
JWT_SECRET=nestfinder_jwt_secret_key_12345
```

backend/package.json

```
{
  "name": "nestfinder-backend",
  "version": "1.0.0",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
  },
  "dependencies": {
    "bcrypt": "^5.1.1",
    "cors": "^2.8.5",
    "dotenv": "^16.4.5",
    "express": "^4.19.2",
    "joi": "^17.13.1",
    "jsonwebtoken": "^9.0.2",
    "mongoose": "^8.4.0",
    "multer": "^1.4.5-lts.1"
  },
  "devDependencies": {
    "nodemon": "^3.1.0"
  }
}
```

backend/models/User.js

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
    email: { type: String, required: true, unique: true },
    password: { type: String, required: true },
    role: { type: String, enum: ['user', 'admin'], default: 'user' }
}, { timestamps: true });

module.exports = mongoose.model('User', userSchema);
```

backend/models/Listing.js

```
const mongoose = require('mongoose');

const listingSchema = new mongoose.Schema({
    title: { type: String, required: true },
    price: { type: Number, required: true },
    location: { type: String, required: true },
    beds: { type: Number, required: true },
```

```
    baths: { type: Number, required: true },
    sqft: { type: Number, required: true },
    tag: { type: String, default: 'Featured Match' },
    image: { type: String, default: '' },
    user: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true }
}, { timestamps: true });

module.exports = mongoose.model('Listing', listingSchema);
```

backend/middlewares/errorMiddleware.js

```
// Global error handling middleware[cite: 1]
const errorHandler = (err, req, res, next) => {
    const statusCode = res.statusCode === 200 ? 500 : res.statusCode;
    res.status(statusCode).json({
        message: err.message || 'Internal Server Error',
        stack: process.env.NODE_ENV === 'production' ? null : err.stack
    });
};

module.exports = errorHandler;
```

backend/middlewares/authMiddleware.js

```
const jwt = require('jsonwebtoken');

// Middleware for client route & API route protection[cite: 1]
const protect = (req, res, next) => {
    let token = req.headers.authorization;

    if (token && token.startsWith('Bearer')) {
        try {
            const decoded = jwt.verify(token.split(' ')[1], process.env.JWT_SECRET);
            req.user = decoded;
            return next();
        } catch (error) {
            return res.status(401).json({ message: 'Not authorized, token failed' });
        }
    }

    return res.status(401).json({ message: 'Not authorized, no token' });
};

module.exports = protect;
```

backend/middlewares/uploadMiddleware.js

```
const multer = require('multer');
const path = require('path');

// Multer middleware for media upload handling[cite: 1]
const storage = multer.diskStorage({
    destination: (req, file, cb) => {
        cb(null, 'uploads/');
    },
    filename: (req, file, cb) => {
        cb(null, `${Date.now()}-${file.originalname}`);
    }
});

const upload = multer({
    storage,
    fileFilter: (req, file, cb) => {
        const ext = path.extname(file.originalname).toLowerCase();
        if ['.png', '.jpg', '.jpeg', '.webp'].includes(ext)) {
            cb(null, true);
        } else {
            cb(new Error('Only image files are allowed'));
        }
    }
});

module.exports = upload;
```

backend/controllers/authController.js

```
const bcrypt = require('bcrypt'); // Password encryption[cite: 1]
const jwt = require('jsonwebtoken'); // Token generation[cite: 1]
const Joi = require('joi'); // JOI validation library[cite: 1]
const User = require('../models/User');
```

```
const authSchema = Joi.object({
    email: Joi.string().email().required(),
    password: Joi.string().min(6).required()
});

exports.register = async (req, res, next) => {
    try {
        const { error } = authSchema.validate(req.body);
        if (error) return res.status(400).json({ message: error.details[0].message });

        const { email, password } = req.body;
        const existingUser = await User.findOne({ email });
        if (existingUser) return res.status(400).json({ message: 'User already exists' });

        const hashedPassword = await bcrypt.hash(password, 10);
        const user = await User.create({ email, password: hashedPassword });

        const token = jwt.sign({ id: user._id }, process.env.JWT_SECRET, { expiresIn: '1d' });
        res.status(201).json({ token, user: { id: user._id, email: user.email } });
    } catch (err) {
        next(err);
    }
};

exports.login = async (req, res, next) => {
    try {
        const { error } = authSchema.validate(req.body);
        if (error) return res.status(400).json({ message: error.details[0].message });

        const { email, password } = req.body;
        const user = await User.findOne({ email });
        if (!user) return res.status(401).json({ message: 'Invalid email or password' });

        const isMatch = await bcrypt.compare(password, user.password);
        if (!isMatch) return res.status(401).json({ message: 'Invalid email or password' });

        const token = jwt.sign({ id: user._id }, process.env.JWT_SECRET, { expiresIn: '1d' });
        res.status(200).json({ token, user: { id: user._id, email: user.email } });
    } catch (err) {
        next(err);
    }
};
```

backend/controllers/listingController.js

```
const Listing = require('../models/Listing');

// Complete CRUD logic[cite: 1]
exports.getListings = async (req, res, next) => {
    try {
        const listings = await Listing.find();
        res.status(200).json(listings);
    } catch (err) {
        next(err);
    }
};

exports.createListing = async (req, res, next) => {
    try {
        const imagePath = req.file ? `/uploads/${req.file.filename}` : '';
        const listing = await Listing.create({
            ...req.body,
            image: imagePath,
            user: req.user.id
        });
        res.status(201).json(listing);
    } catch (err) {
        next(err);
    }
};

exports.updateListing = async (req, res, next) => {
    try {
        const updated = await Listing.findByIdAndUpdate(req.params.id, req.body, { new: true });
        res.status(200).json(updated);
    } catch (err) {
        next(err);
    }
};
```

```
exports.deleteListing = async (req, res, next) => {
    try {
        await Listing.findByIdAndDelete(req.params.id);
        res.status(200).json({ message: 'Listing removed successfully' });
    } catch (err) {
        next(err);
    }
};
```

backend/routes/authRoutes.js

```
const express = require('express');
const router = express.Router();
const { register, login } = require('../controllers/authController');

router.post('/register', register);
router.post('/login', login);

module.exports = router;
```

backend/routes/listingRoutes.js

```
const express = require('express');
const router = express.Router();
const protect = require('../middlewares/authMiddleware');
const upload = require('../middlewares/uploadMiddleware');
const { getListings, createListing, updateListing, deleteListing } = require('../controllers/listingController');

router.get('/', getListings);
router.post('/', protect, upload.single('image'), createListing);
router.put('/:id', protect, updateListing);
router.delete('/:id', protect, deleteListing);

module.exports = router;
```

backend/server.js

```
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
const path = require('path');
require('dotenv').config();

const authRoutes = require('./routes/authRoutes');
const listingRoutes = require('./routes/listingRoutes');
const errorHandler = require('./middlewares/errorMiddleware');

const app = express();

app.use(cors());
app.use(express.json());
app.use('/uploads', express.static(path.join(__dirname, 'uploads')));

app.use('/api/auth', authRoutes);
app.use('/api/listings', listingRoutes);

app.use(errorHandler);

const PORT = process.env.PORT || 5000;

mongoose.connect(process.env.MONGO_URI)
    .then(() => {
        console.log('MongoDB Connected');
        app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
    })
    .catch((err) => console.error('MongoDB Connection Error:', err));
```

frontend/package.json

```
{
  "name": "nestfinder-frontend",
  "private": true,
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "@reduxjs/toolkit": "^2.2.5",
```

```
    "axios": "^1.7.2",
    "react": "^18.3.1",
    "react-dom": "^18.3.1",
    "react-redux": "^9.1.2",
    "react-router-dom": "^6.23.1"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "^4.3.0",
    "vite": "^5.2.11"
  }
}
```

frontend/src/main.jsx

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import { Provider } from 'react-redux';
import { BrowserRouter } from 'react-router-dom';
import { store } from './redux/store';
import { AuthProvider } from './context/authContext';
import App from './App';

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <Provider store={store}>
      <AuthProvider>
        <BrowserRouter>
          <App />
        </BrowserRouter>
      </AuthProvider>
    </Provider>
  </React.StrictMode>
);
```

frontend/src/App.jsx

```
import React from 'react';
import { Routes, Route } from 'react-router-dom';
import Homepage from './components/homepage';
import Login from './components/login';
import Register from './components/Register';
import ProtectedRoute from './components/ProtectedRoute';

function App() {
  return (
    <Routes>
      <Route path="/" element={<Homepage />} />
      <Route path="/login" element={<Login />} />
      <Route path="/register" element={<Register />} />
      <Route
        path="/dashboard"
        element={
          <ProtectedRoute>
            <div style={{ padding: '4rem', textAlign: 'center' }}>
              <h2>User Dashboard</h2>
              <p>Welcome to your protected NestFinder dashboard!</p>
            </div>
          </ProtectedRoute>
        }
      />
    </Routes>
  );
}

export default App;
```

frontend/src/components/Register.jsx

```
import React, { useState } from 'react';
import axios from 'axios';
import { useNavigate } from 'react-router-dom';
import { useAuthContext } from '../context/authContext';

const Register = () => {
    const [email, setEmail] = useState('');
    const [password, setPassword] = useState('');
    const [error, setError] = useState('');
    const [isLoading, setIsLoading] = useState(false);
    const { loginUser } = useAuthContext();
    const navigate = useNavigate();
```

```
    const handleRegister = async (e) => {
        e.preventDefault();
        if (!email || !password) return setError('All fields are required');

        setIsLoading(true);
        try {
            const response = await axios.post('http://localhost:5000/api/auth/register', { email, password });
            loginUser(response.data.token, response.data.user);
            navigate('/');
        } catch (err) {
            setError(err.response?.data?.message || 'Registration failed');
        } finally {
            setIsLoading(false);
        }
    };

    return (
        <div style={{ maxWidth: '400px', margin: '4rem auto', padding: '2rem', border: '1px solid #e2e8f0', borderRa
            <h2>Create NestFinder Account</h2>
            {error && <p style={{ color: 'red' }}>{error}</p>}
            <form onSubmit={handleRegister}>
                <div style={{ marginBottom: '1rem' }}>
                    <label>Email</label>
                    <input type="email" value={email} onChange={(e) => setEmail(e.target.value)} required style={{ w
                </div>
                <div style={{ marginBottom: '1rem' }}>
                    <label>Password</label>
                    <input type="password" value={password} onChange={(e) => setPassword(e.target.value)} required s
                </div>
                <button type="submit" disabled={isLoading} style={{ width: '100%', padding: '0.75rem', background: '
                    {isLoading ? 'Creating Account...' : 'Register'}
                </button>
            </form>
        </div>
    );
};

export default Register;
```

frontend/src/components/ProtectedRoute.jsx

```
import React from 'react';
import { Navigate } from 'react-router-dom';
import { useAuthContext } from '../context/authContext';

// Client-side protected route wrapper[cite: 1]
const ProtectedRoute = ({ children }) => {
    const { userToken } = useAuthContext();

    if (!userToken) {
        return <Navigate to="/login" replace />;
    }

    return children;
};

export default ProtectedRoute;
```

User prompt: Explain what each code does, and how should I implement it

Response: Transitioning from standalone files to a full-stack architecture is a massive leap, but you are setting this up perfectly! The files we generated divide your application into two distinct halves: the server (backend) and the user interface (frontend).

Here is a straightforward breakdown of what these files do and how to bring your application to life.

The Backend (Node.js & Express)

These files act as the brain of your application, securely handling data, user verification, and database communication.

- **`server.js`, `package.json`, & `.env`:** `server.js` is the engine that starts your backend, connects to MongoDB, and listens for requests. The `.env` file hides sensitive data (like passwords and database links), while `package.json` tracks all the downloaded packages required to run the server.
- **`models/` (User.js, Listing.js):** These define the exact blueprint of your database. They ensure that every user has an email and password, and every house listing has a price, location, and image.
- **`controllers/` & `routes/`:** Routes are the specific URLs the frontend calls (e.g., `/api/auth/login`). Controllers hold the actual logic for those URLs—such as hashing a password, checking a database, and issuing a secure token.

- **`middlewares/`:** These are security checkpoints. `authMiddleware.js` blocks guests from seeing private data, `errorMiddleware.js` catches crashes gracefully, and `uploadMiddleware.js` specifically handles saving image files sent from the frontend.

The Frontend (React SPA)

These files create the interactive user interface that runs inside the user's browser.

- **`main.jsx` & `package.json`:** `main.jsx` is the entry point that injects your React app into the browser. It also wraps your app in the Redux and Context providers so data flows seamlessly everywhere.
- **`App.jsx`:** This is your traffic controller. It uses React Router to swap between the Homepage, Login, and Register screens instantly without reloading the browser window.
- **`Register.jsx`:** A standard form component that collects user data and uses Axios to send a creation request to your backend.
- **`ProtectedRoute.jsx`:** A client-side security guard. If a user tries to access the `/dashboard` URL without being logged in, this file instantly kicks them back to the login screen.

Step-by-Step Implementation Guide

To implement this, you need to run both sides of your application simultaneously using your computer's terminal.

1. **Prepare the Database:** Ensure you have **MongoDB** installed and running on your computer (or use a free MongoDB Atlas cloud cluster and update the `MONGO_URI` in your `.env` file).
2. **Start the Backend:** Open a terminal, navigate into your `backend` folder, and install the dependencies. Then, start the development server:

   ```
   cd backend
   npm install
   npm run dev
   ```

   You should see "MongoDB Connected" and "Server running on port 5000".
3. **Start the Frontend:** Open a *second, separate* terminal window, navigate into your `frontend` folder, install its dependencies, and start the React app:

   ```
   cd frontend
   npm install
   npm run dev
   ```

   This will give you a local host link (usually `http://localhost:5173`) to view your live, clicking website.

Are you ready to test creating a user account through the new React interface, or would you like to set up dummy data in the database first?

User prompt: I have a free MongoDB Atlas cloud cluster open. What should I do in it?

Response: Setting up a MongoDB Atlas cluster is a perfect move, especially since your final project instructions require you to deploy your application to the cloud later on. A local database won't work once your backend is on Heroku, so you are getting ahead of the game!

To connect your Atlas cluster to your backend, you need to configure access permissions and grab your unique connection string. Here is exactly what to do inside your MongoDB Atlas dashboard:

1. Allow Network Access (IP Whitelisting)

By default, Atlas blocks all outside connections. You need to tell it to accept connections from your server.

1. On the left sidebar, under the **Security** section, click on **Network Access**.
2. Click the green **+ Add IP Address** button.
3. For development, click **Allow Access from Anywhere** (this populates `0.0.0.0/0`). *Note: In a strict production environment, you would only whitelist your specific server IPs, but this is best for a student project.*
4. Click **Confirm** and wait a moment for the status to turn to "Active".

2. Create a Database User

You need a specific username and password for your application to log into the database.

1. On the left sidebar, click on **Database Access**.
2. Click the green **+ Add New Database User** button.
3. Select **Password** as the Authentication Method.
4. Create a **Username** (e.g., `nestfinder_admin`) and a **Password**.
5. *Important:* Write this password down or copy it somewhere temporarily. You will need it in a minute. Do not use special characters like `@` or `:` in the password to avoid breaking the URL.
6. Click **Add User** at the bottom.

3. Get Your Connection String

Now you need the exact URL that Node.js will use to talk to your cluster.

1. On the left sidebar, under the **Deployment** section, click on **Database**.
2. You should see your cluster (usually named `Cluster0`). Click the **Connect** button next to it.
3. Select **Drivers** (since we are using Node.js).
4. You will see a connection string that looks something like this: `mongodb+srv://<username>:<password>@cluster0.abcde.mongodb.net/?retryWrites=true&w=majority&appName=Cluster0`
5. **Copy** that string.

4. Update Your Backend `.env` File

Go back to your code editor, open your `backend/.env` file, and replace the old local URI with your new cloud URI.

You need to make two manual edits to the string you pasted:

- Replace `<password>` with the actual database user password you created in Step 2.
- Add a database name (like `nestfinder`) right before the `?` in the URL.

Your `.env` file should now look like this:

```
PORT=5000
MONGO_URI=mongodb+srv://nestfinder_admin:YourSecretPassword123@cluster0.abcde.mongodb.net/nestfinder?retryWrites=tru
JWT_SECRET=nestfinder_jwt_secret_key_12345
```

Once you save that file, restart your backend server in the terminal (`Ctrl+C` to stop it, then `npm run dev` to start it again).

Does the terminal successfully print **"MongoDB Connected"** with your new cloud database?

User prompt: What IP address should I type in/

Response: Type `0.0.0.0/0` into the **Access List Entry** box.

This specific CIDR notation tells MongoDB Atlas to allow connections from *any* IP address. This is the best approach for your project because you are currently developing locally, and later you are required to deploy your backend to Heroku. Heroku uses dynamic IP addresses that change constantly, so restricting the database to a single IP will block Heroku and cause your deployed app to crash.

For the **Comment** box, you can simply type "Anywhere" or "Heroku Deployment", and then click the green **Confirm** button.

Did the status indicator switch to a green "Active" after you confirmed the entry?

User prompt: I have this existing user, it was made by the server itself. Do I use it or make a new user anyway?

Response: You can definitely use `peter13gs_db_user` since it already has full access permissions (`atlasAdmin@admin`).

- **If you know the password:** Use `peter13gs_db_user` and that password directly inside your `MONGO_URI` connection string.
- **If you don't know the password:** Click **EDIT** on the right side of that row, select **Edit Password**, enter a new password, and save it.
- **If you prefer a clean setup:** Click **+ ADD NEW DATABASE USER** in the top-right corner to create a dedicated user (e.g., `nestfinder_admin`).

Either choice works identically for Node.js and Mongoose. Once you have a username and password pair you remember, you are ready to copy the connection string from the **Database** tab into your `backend/.env` file.

User prompt: Where are the deployment settings?

Response: You are already on the correct page in MongoDB Atlas (`DATABASE` > `Clusters`).

Depending on what specific settings you need:

- **To get your connection string (for `.env`):** Click the **Connect** button directly next to `Cluster0` in the middle of your screen.
- **To change cluster settings (Region, Instance Size, Scaling):** Click the **Edit Config** button in the top-right corner of the `Cluster0` box, or click the `...` button next to *Browse Collections*.

  Note: MongoDB Atlas only hosts your **database**. To deploy your actual application code (Node.js backend and React frontend), you will deploy the backend to a cloud host like Render/Heroku and the frontend to Vercel or Netlify.

User prompt: Fix

Response: Based on the error message in "image_761cc4.jpg", I see exactly what is causing this. This is a very common error when working with React on a Windows computer.

The issue is a **casing mismatch**. In some of your files, you are importing the file as `authContext` (lowercase 'a'), and in others, you are importing it as `AuthContext` (capital 'A'). While Windows ignores case differences in filenames, React's bundler (Vite) is strictly case-sensitive and thinks you are trying to import two completely different files.

Here is how to fix it quickly and permanently:

Step 1: Standardize the actual file name

First, look inside your `frontend/src/context/` folder. Make sure the actual file is explicitly named with a capital A: **`AuthContext.jsx`**. *(Note: If it is already named this, you can skip to Step 2).*

Step 2: Fix the imports across your app

You need to do a quick search through your frontend files and update the import paths so that every single one uses the exact same capital 'A'.

Check the following files and update the import line to look exactly like this:

In `frontend/src/main.jsx`: Change: `import { AuthProvider } from './context/authContext';` To: `import { AuthProvider } from './context/AuthContext';`

In `frontend/src/components/Register.jsx`: Change: `import { useAuthContext } from '../context/authContext';` To: `import { useAuthContext } from '../context/AuthContext';`

In `frontend/src/components/ProtectedRoute.jsx`: Change: `import { useAuthContext } from '../context/authContext';` To: `import { useAuthContext } from '../context/AuthContext';`

Step 3: Restart your server

Sometimes VS Code and Vite hold onto the old cached names even after you fix them.

1. Go to your frontend terminal.
2. Stop the server by pressing `Ctrl + C`.
3. Start it again by typing `npm run dev`.

The red squiggly lines and the terminal error should instantly disappear! Let me know if it pops up anywhere else.

User prompt: This is what happens when I try to run the project. Why does it happen and how do I fix it?

Response: I can certainly help you figure out why that warning is appearing in **image_af6ee7.png** and how to get rid of it.

Why This Happens

The warning in your terminal occurs because Vite (the build tool powering your development server) relies on an `index.html` file to act as the main entry point for your application. When you run `npm run dev`, Vite scans this HTML file to discover your JavaScript or TypeScript files and pre-bundles your dependencies to make the server load incredibly fast.

If Vite cannot find the `index.html` file in the root directory where the command was executed (in your case, the `frontend` folder), it throws this warning and skips the dependency pre-bundling process.

How to Fix It

This is a very common issue, especially for developers who are used to older React setups. Here is how you can resolve it:

- **Move `index.html` to the project root:** Unlike Create React App (which keeps the HTML file in a `public/` folder), Vite requires `index.html` to be at the absolute root of your project. Ensure the file is sitting directly inside your `frontend` folder, right next to your `package.json` and `vite.config.js` files.
- **Fix the script path:** If you had to move the `index.html` file to the root, you will likely need to update the script tag inside it so it can find your main code. Ensure it points to your source directory, like this: `<script type="module" src="/src/main.jsx"></script>` (or `.tsx`/`.js` depending on your setup).
- **Verify your terminal directory:** Check if your actual project files are nested one folder deeper. If your code is actually inside something like `frontend/my-vite-app`, you need to run `cd my-vite-app` before running `npm run dev`.
- **Configure a custom root (Advanced):** If you specifically want your HTML file to live in a different directory (like `src/`), you must explicitly tell Vite by updating your `vite.config.js` file and setting the `root: 'src'` property.

Did you recently migrate this project from Create React App or a Webpack environment?

User prompt: I have made index.html according to your instructions, and the page I get when running the project is still blank. What might've caused it and how do I fix it?

Response: It can be incredibly frustrating to fire up a project only to be greeted by a blank screen! Fortunately, looking at the code you provided, the culprit is a very common and easy-to-fix typo.

The issue lies within your index.html file. In the `<body>` of your document, you have a `<div>` defined with `class="root"`.

Most JavaScript frameworks (like React, which you seem to be using based on the `main.jsx` file) mount the application by looking for a specific HTML element using its **ID**, typically with a command like `document.getElementById('root')`. Because your container currently uses a class instead of an ID, the JavaScript cannot find where to inject the app, resulting in a blank page.

The Fix

To resolve this, simply change the `class` attribute to an `id` attribute.

Change this line: `<div class="It looks like there is a small typo in your HTML that is preventing your app from rendering.

The Cause

In your "index.html" file, your main container is written as `<div class="root">`.

In a standard React or Vite setup (which is indicated by your `/src/main.jsx` script), the application expects to mount into an element with an **ID** of "root", not a class. Your `main.jsx` file is likely running `document.getElementById('root')`. Because it cannot find an element with that ID, it fails to mount the application, resulting in a completely blank page.

The Fix

To fix this, you just need to change the `class` attribute to an `id` attribute. Update your "index.html" to look like this:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
</head>
<body>
    <!-- Change 'class' to 'id' here -->
    <div id="root">

    </div>
    <script type="module" src="/src/main.jsx"></script>
</body>
</html>
```

Save the file and refresh your browser.

Did changing the class to an ID resolve the blank page, or are you seeing any new red error messages in your browser's developer console?

User prompt: Generate homepage.css for me

Response:
```
/* Reset & Base Styles */
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
    font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Helvetica, Arial, sans-serif;
}

body {
    background-color: #f8fafc;
    color: #0f172a;
    line-height: 1.5;
}

.homepage {
    width: 100%;
    min-height: 100vh;
}

/* Header & Navigation */
header {
    background-color: #ffffff;
    box-shadow: 0 1px 3px rgba(0, 0, 0, 0.05);
    position: sticky;
    top: 0;
    z-index: 100;
}

.nav-container {
```

```
        max-width: 1200px;
        margin: 0 auto;
        padding: 1.25rem 2rem;
        display: flex;
        justify-content: space-between;
        align-items: center;
    }

    .logo {
        font-size: 1.5rem;
        font-weight: 700;
        color: #2563eb;
        text-decoration: none;
        display: flex;
        align-items: center;
        gap: 0.5rem;
    }

    .nav-links {
        display: flex;
        list-style: none;
        gap: 2rem;
        align-items: center;
    }

    .nav-links a {
        text-decoration: none;
        color: #475569;
        font-weight: 500;
        transition: color 0.2s ease;
    }

    .nav-links a:hover {
        color: #2563eb;
    }

    .btn-primary {
        background-color: #2563eb;
        color: #ffffff !important;
        padding: 0.6rem 1.25rem;
        border-radius: 8px;
        transition: background-color 0.2s ease !important;
    }

    .btn-primary:hover {
        background-color: #1d4ed8;
    }

    /* Hero Section */
    .hero {
        background: linear-gradient(135deg, #1e293b 0%, #0f172a 100%);
        color: #ffffff;
        padding: 5rem 2rem 6rem;
        text-align: center;
    }

    .hero-content {
        max-width: 900px;
        margin: 0 auto;
    }

    .hero-content h1 {
        font-size: 3rem;
        font-weight: 800;
        margin-bottom: 1rem;
        line-height: 1.2;
    }

    .hero-content h1 span {
        color: #38bdf8;
    }

    .hero-content p {
        font-size: 1.2rem;
        color: #94a3b8;
        margin-bottom: 3rem;
        max-width: 650px;
        margin-left: auto;
        margin-right: auto;
```

```
}

/* Search Bar Container */
.search-container {
    background-color: #ffffff;
    border-radius: 12px;
    padding: 1.25rem;
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(160px, 1fr)) auto;
    gap: 1rem;
    box-shadow: 0 10px 25px -5px rgba(0, 0, 0, 0.3);
    align-items: end;
}

.search-group {
    display: flex;
    flex-direction: column;
    text-align: left;
}

.search-group label {
    font-size: 0.85rem;
    font-weight: 600;
    color: #64748b;
    margin-bottom: 0.4rem;
    text-transform: uppercase;
    letter-spacing: 0.05em;
}

.search-group input,
.search-group select {
    width: 100%;
    padding: 0.65rem 0.75rem;
    border: 1px solid #cbd5e1;
    border-radius: 6px;
    font-size: 0.95rem;
    color: #0f172a;
    outline: none;
    background-color: #ffffff;
}

.search-group input:focus,
.search-group select:focus {
    border-color: #2563eb;
    box-shadow: 0 0 0 2px rgba(37, 99, 235, 0.2);
}

.search-btn {
    background-color: #2563eb;
    color: #ffffff;
    border: none;
    border-radius: 6px;
    padding: 0.75rem 1.5rem;
    font-weight: 600;
    cursor: pointer;
    display: flex;
    align-items: center;
    gap: 0.5rem;
    justify-content: center;
    transition: background-color 0.2s ease;
    height: 42px;
}

.search-btn:hover {
    background-color: #1d4ed8;
}

/* General Section Layout */
.section {
    max-width: 1200px;
    margin: 0 auto;
    padding: 5rem 2rem;
}

.section-header {
    text-align: center;
    margin-bottom: 3rem;
}
```

```
    .section-header h2 {
        font-size: 2.25rem;
        font-weight: 700;
        color: #0f172a;
        margin-bottom: 0.5rem;
    }

    .section-header p {
        color: #64748b;
        font-size: 1.1rem;
    }

    /* Features Grid */
    .features-grid {
        display: grid;
        grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
        gap: 2rem;
    }

    .feature-card {
        background-color: #ffffff;
        padding: 2.5rem 2rem;
        border-radius: 12px;
        border: 1px solid #e2e8f0;
        transition: transform 0.2s ease, box-shadow 0.2s ease;
    }

    .feature-card:hover {
        transform: translateY(-4px);
        box-shadow: 0 10px 20px rgba(0, 0, 0, 0.05);
    }

    .feature-icon {
        width: 50px;
        height: 50px;
        background-color: #eff6ff;
        color: #2563eb;
        border-radius: 10px;
        display: flex;
        align-items: center;
        justify-content: center;
        font-size: 1.5rem;
        margin-bottom: 1.5rem;
    }

    .feature-card h3 {
        font-size: 1.25rem;
        font-weight: 600;
        margin-bottom: 0.75rem;
    }

    .feature-card p {
        color: #64748b;
        font-size: 0.95rem;
        line-height: 1.6;
    }

    /* Listings Grid */
    .listings-grid {
        display: grid;
        grid-template-columns: repeat(auto-fill, minmax(320px, 1fr));
        gap: 2rem;
    }

    .listing-card {
        background-color: #ffffff;
        border-radius: 12px;
        overflow: hidden;
        border: 1px solid #e2e8f0;
        transition: transform 0.2s ease, box-shadow 0.2s ease;
    }

    .listing-card:hover {
        transform: translateY(-4px);
        box-shadow: 0 12px 24px rgba(0, 0, 0, 0.08);
    }

    .listing-img {
        height: 220px;
```

```
        background-size: cover;
        background-position: center;
        position: relative;
        padding: 1rem;
    }

    .tag {
        background-color: rgba(15, 23, 42, 0.8);
        backdrop-filter: blur(4px);
        color: #ffffff;
        padding: 0.35rem 0.75rem;
        border-radius: 20px;
        font-size: 0.75rem;
        font-weight: 600;
        text-transform: uppercase;
    }

    .listing-content {
        padding: 1.5rem;
    }

    .price {
        font-size: 1.5rem;
        font-weight: 700;
        color: #2563eb;
        margin-bottom: 0.25rem;
    }

    .listing-title {
        font-size: 1.1rem;
        font-weight: 600;
        margin-bottom: 0.5rem;
        color: #0f172a;
    }

    .location {
        color: #64748b;
        font-size: 0.9rem;
        margin-bottom: 1.25rem;
        display: flex;
        align-items: center;
        gap: 0.35rem;
    }

    .details {
        display: flex;
        justify-content: space-between;
        padding-top: 1rem;
        border-top: 1px solid #f1f5f9;
        color: #475569;
        font-size: 0.85rem;
        font-weight: 500;
    }

    .details span {
        display: flex;
        align-items: center;
        gap: 0.35rem;
    }

    /* Status States */
    .loading-state,
    .error-state {
        text-align: center;
        font-size: 1.1rem;
        padding: 2rem;
        grid-column: 1 / -1;
    }

    .loading-state {
        color: #2563eb;
    }

    .error-state {
        color: #ef4444;
    }

    /* Footer */
    footer {
```

```
    background-color: #0f172a;
    color: #94a3b8;
    padding: 4rem 2rem 2rem;
    margin-top: 4rem;
}

.footer-container {
    max-width: 1200px;
    margin: 0 auto;
    display: flex;
    justify-content: space-between;
    align-items: center;
    padding-bottom: 3rem;
    border-bottom: 1px solid #1e293b;
}

.footer-info .logo {
    color: #ffffff;
    margin-bottom: 0.75rem;
}

.footer-info p {
    font-size: 0.95rem;
}

.copyright {
    text-align: center;
    padding-top: 2rem;
    font-size: 0.85rem;
    color: #64748b;
}

/* Responsive Styles */
@media (max-width: 768px) {
    .hero-content h1 {
        font-size: 2.25rem;
    }

    .search-container {
        grid-template-columns: 1fr;
    }

    .footer-container {
        flex-direction: column;
        text-align: center;
        gap: 1.5rem;
    }

    .footer-info .logo {
        justify-content: center;
    }
}
```

User prompt: Help me alter the homepage- I have decided to change the purpose of the project and turn it into a website for people looking to get out of a toxic household and people willing to offer them a place to stay. i have already made changes, but I need help making further changes to fit my new vision

Response: This is a profound and incredibly meaningful pivot for your project. Shifting from a standard real estate app to a mutual aid platform for people escaping toxic environments adds a layer of social impact that will make this portfolio piece stand out immediately.

To make this work, the UI needs to shift from a tone of "commercial real estate" to one of "safety, urgency, and community trust." The focus must move away from mortgages and property taxes, and instead highlight verification, privacy, and immediate availability.

Here is the fully updated `homepage.jsx` tailored to your new vision, keeping your existing Redux and Context logic intact so your backend connection does not break.

```
import React, { useEffect } from 'react';
import { useDispatch, useSelector } from 'react-redux';
import { fetchListings } from '../redux/listingsSlice';
import { useAuthContext } from '../context/AuthContext';
import './homepage.css';

const Homepage = () => {
    const dispatch = useDispatch();
    const { userToken } = useAuthContext();
    const { items: listings, status, error } = useSelector((state) => state.listings);

    useEffect(() => {
```

```
    if (status === 'idle') {
        dispatch(fetchListings());
    }
}, [status, dispatch]);

return (
    <div className="homepage">
        <header>
            <nav className="nav-container">
                <a href="#" className="logo">
                    <i className="fa-solid fa-hands-holding-circle"></i> NestFinder
                </a>
                <ul className="nav-links">
                    <li><a href="#features">Our Mission</a></li>
                    <li><a href="#listings">Find Safe Spaces</a></li>
                    <li>{userToken ? <a href="/dashboard" className="btn-primary">Dashboard</a> : <a href="/logi
                </ul>
            </nav>
        </header>

        <section className="hero">
            <div className="hero-content">
                <h1>Find a <span>Safe Place</span> to Land</h1>
                <p>Our goal is to match you with a space where you will actually thrive, and get you out of a ne

                <div className="search-container">
                    <div className="search-group">
                        <label>Location</label>
                        <input type="text" placeholder="City, State, or Zip" defaultValue="Austin, TX" />
                    </div>
                    <div className="search-group">
                        <label>Space Type</label>
                        <select defaultValue="Spare Room">
                            <option>Spare Room</option>
                            <option>Living Room Couch</option>
                            <option>In-law Suite</option>
                            <option>Entire Apartment</option>
                        </select>
                    </div>
                    <div className="search-group">
                        <label>Arrangement</label>
                        <select defaultValue="Emergency (Free)">
                            <option>Emergency (Free)</option>
                            <option>Low-Cost Rent</option>
                            <option>Work/Chore Exchange</option>
                        </select>
                    </div>
                    <div className="search-group">
                        <label>Urgency</label>
                        <select defaultValue="Immediate">
                            <option>Immediate / Tonight</option>
                            <option>Within a week</option>
                            <option>Planning ahead</option>
                        </select>
                    </div>
                    <button className="search-btn">
                        <i className="fa-solid fa-magnifying-glass"></i> Search
                    </button>
                </div>
            </div>
        </section>

        <section className="section" id="features">
            <div className="section-header">
                <h2>Safety and Support First</h2>
                <p>Leaving a difficult environment is hard enough. Finding a safe, supportive place to rest shou
            </div>
            <div className="features-grid">
                <div className="feature-card">
                    <div className="feature-icon"><i className="fa-solid fa-shield-halved"></i></div>
                    <h3>Verified Hosts</h3>
                    <p>Every host offering a space goes through a community verification process and background
                </div>
                <div className="feature-card">
                    <div className="feature-icon"><i className="fa-solid fa-user-lock"></i></div>
                    <h3>Absolute Privacy</h3>
                    <p>Your safety is our priority. Exact addresses and personal contact information are kept st
                </div>
                <div className="feature-card">
```

```
                        <div className="feature-icon"><i className="fa-solid fa-handshake-angle"></i></div>
                        <h3>Clear Boundaries</h3>
                        <p>Our platform helps you and your host establish clear expectations, duration of stay, and
                    </div>
                </div>
            </section>

            <section className="section listings-section" id="listings">
                <div className="section-header">
                    <h2>Welcoming Hosts Near You</h2>
                    <p>Generous community members currently offering a safe environment.</p>
                </div>

                {status === 'loading' && <p className="loading-state">Loading available spaces...</p>}
                {error && <p className="error-state">Failed to load spaces: {error}</p>}

                <div className="listings-grid">
                    {listings && listings.map((listing) => (
                        <div className="listing-card" key={listing._id || listing.id}>
                            <div className="listing-img" style={{ backgroundImage: `url(${listing.image || 'https://
                                <span className="tag">{listing.price === 0 || listing.price === '0' ? 'Free / Emerge
                            </div>
                            <div className="listing-content">
                                <div className="price">
                                    {listing.price === 0 || listing.price === '0' ? 'No Cost' : `$${listing.price?.t
                                </div>
                                <h3 className="listing-title">{listing.title}</h3>
                                <div className="location"><i className="fa-solid fa-location-dot"></i> {listing.loca
                                <div className="details">
                                    <span><i className="fa-solid fa-bed"></i> {listing.beds} Bed(s)</span>
                                    <span><i className="fa-solid fa-bath"></i> {listing.baths} Bath</span>
                                    {/* We keep sqft if it exists in your DB, or we can use it to denote max duratio
                                    <span><i className="fa-solid fa-calendar-day"></i> {listing.sqft} Days Max</span
                                </div>
                            </div>
                        </div>
                    ))}
                </div>
            </section>

            <footer>
                <div className="footer-container">
                    <div className="footer-info">
                        <a href="#" className="logo">
                            <i className="fa-solid fa-hands-holding-circle"></i> NestFinder
                        </a>
                        <p>Helping you transition safely into a space where you can heal and rebuild.</p>
                    </div>
                </div>
                <div className="copyright">
                    &copy; 2026 NestFinder Mutual Aid. All rights reserved.
                </div>
            </footer>
        </div>
    );
};

export default Homepage;
```

Key Functional Changes:

- **Search Parameters:** Changed from purely real estate metrics (Price, School Districts) to crisis-intervention metrics (Space Type, Arrangement, Urgency).
- **Feature Cards:** Replaced the financial tools (TrueCost Calculator) with trust and safety pillars: **Verified Hosts**, **Absolute Privacy**, and **Clear Boundaries**.
- **Dynamic Data Handling:** In the Listings map, I altered how your database numbers render. If a host lists the `price` as `0`, it now renders as "No Cost" or "Free / Emergency". I also temporarily repurposed the `sqft` database field to represent "Days Max" to make it more relevant to a temporary stay without requiring you to rewrite your backend Mongoose models right now.

To make the icons match the new theme, you may want to ensure you are pulling in FontAwesome in your `index.html`. Let me know if you would like help adjusting the backend database models to reflect these new concepts!

User prompt: Yes, help me adjust the existing pages to the new purpose. Suggest and explain alterations

Response: To fully realize this pivot, the most critical changes need to happen in your database structures and user onboarding. A mutual aid platform requires distinct user roles (those offering space vs. those seeking it) and a completely different set of data points for the spaces being

offered.

Here are the specific alterations to your existing files and the reasoning behind them.

1. Database Models (`backend/models/`)

We need to change what information is saved to the database. Real estate focuses on permanent features (square footage, property taxes), whereas emergency housing focuses on duration, type of accommodation, and safety rules.

Update `backend/models/User.js` We must distinguish between a "Host" and a "Seeker" at registration so you can show them different dashboards later.

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
    email: { type: String, required: true, unique: true },
    password: { type: String, required: true },
    // Replaced standard 'user/admin' with purpose-driven roles
    role: { type: String, enum: ['seeker', 'host', 'admin'], required: true },
    // Added for future trust & safety features
    isVerified: { type: Boolean, default: false }
}, { timestamps: true });

module.exports = mongoose.model('User', userSchema);
```

Update `backend/models/Listing.js` We will repurpose the listing model from a "Property for Sale" to a "Space for Offer."

```
const mongoose = require('mongoose');

const listingSchema = new mongoose.Schema({
    title: { type: String, required: true }, // e.g., "Quiet Spare Room in Suburbs"
    location: { type: String, required: true }, // Should just be City/Neighborhood for safety, not full address
    spaceType: { type: String, required: true }, // e.g., "Couch", "Private Room", "In-law Suite"
    price: { type: Number, default: 0 }, // Often 0, or a small contribution
    beds: { type: Number, required: true },
    baths: { type: Number, required: true },
    maxDuration: { type: Number, required: true }, // Replacing sqft: Maximum days host can accommodate
    rules: { type: String, default: 'No smoking, pet friendly' }, // Crucial for establishing boundaries
    tag: { type: String, default: 'Emergency / Free' },
    image: { type: String, default: '' },
    user: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true }
}, { timestamps: true });

module.exports = mongoose.model('Listing', listingSchema);
```

2. Backend Authentication Controller (`backend/controllers/authController.js`)

Since we added a `role` to the User model, the registration logic needs to accept and validate this new field so it gets saved to the database.

Update the `authSchema` validation block at the top of the file:

```
const authSchema = Joi.object({
    email: Joi.string().email().required(),
    password: Joi.string().min(6).required(),
    role: Joi.string().valid('seeker', 'host').required() // Force user to pick a path
});
```

Update the user creation line inside the `exports.register` function:

```
// Add role to the destructured body and the creation object
const { email, password, role } = req.body;
// ... (existing existingUser check)
const hashedPassword = await bcrypt.hash(password, 10);
const user = await User.create({ email, password: hashedPassword, role });
```

3. Frontend Registration (`frontend/src/components/Register.jsx`)

The registration page must now reflect the new mission. The tone should be supportive, and it must include a dropdown allowing the user to select if they need help or are offering help.

```
import React, { useState } from 'react';
import axios from 'axios';
import { useNavigate, Link } from 'react-router-dom';
import { useAuthContext } from '../context/AuthContext';

const Register = () => {
    const [email, setEmail] = useState('');
    const [password, setPassword] = useState('');
    const [role, setRole] = useState('seeker'); // Default to seeker
    const [error, setError] = useState('');
    const [isLoading, setIsLoading] = useState(false);
```

```
    const { loginUser } = useAuthContext();
    const navigate = useNavigate();

    const handleRegister = async (e) => {
        e.preventDefault();
        if (!email || !password || !role) return setError('All fields are required');

        setIsLoading(true);
        try {
            const response = await axios.post('http://localhost:5000/api/auth/register', { email, password, role });
            loginUser(response.data.token, response.data.user);
            navigate('/dashboard'); // Route to dashboard after success
        } catch (err) {
            setError(err.response?.data?.message || 'Registration failed');
        } finally {
            setIsLoading(false);
        }
    };

    return (
        <div style={{ maxWidth: '450px', margin: '4rem auto', padding: '2.5rem', border: '1px solid #e2e8f0', border
            <h2 style={{ marginBottom: '0.5rem', color: '#0f172a' }}>Join the NestFinder Community</h2>
            <p style={{ color: '#64748b', marginBottom: '2rem', fontSize: '0.95rem' }}>A safe space is just a few st

            {error && <div style={{ backgroundColor: '#fee2e2', color: '#ef4444', padding: '0.75rem', borderRadius:

            <form onSubmit={handleRegister}>
                <div style={{ marginBottom: '1.25rem' }}>
                    <label style={{ display: 'block', marginBottom: '0.5rem', fontWeight: '500', fontSize: '0.9rem'
                    <select value={role} onChange={(e) => setRole(e.target.value)} style={{ width: '100%', padding:
                        <option value="seeker">Find a safe place to stay</option>
                        <option value="host">Offer a space to someone in need</option>
                    </select>
                </div>
                <div style={{ marginBottom: '1.25rem' }}>
                    <label style={{ display: 'block', marginBottom: '0.5rem', fontWeight: '500', fontSize: '0.9rem'
                    <input type="email" value={email} onChange={(e) => setEmail(e.target.value)} required style={{ w
                </div>
                <div style={{ marginBottom: '1.5rem' }}>
                    <label style={{ display: 'block', marginBottom: '0.5rem', fontWeight: '500', fontSize: '0.9rem'
                    <input type="password" value={password} onChange={(e) => setPassword(e.target.value)} required m
                </div>
                <button type="submit" disabled={isLoading} style={{ width: '100%', padding: '0.85rem', background: '
                    {isLoading ? 'Creating Account...' : 'Create Account'}
                </button>
            </form>
            <p style={{ marginTop: '1.5rem', textAlign: 'center', fontSize: '0.9rem', color: '#64748b' }}>
                Already have an account? <Link to="/login" style={{ color: '#2563eb', textDecoration: 'none', fontWe
            </p>
        </div>
    );
};

export default Register;
```

With these models updated, how would you like to structure the user Dashboard? We could branch it into two different views: one for Hosts to manage the spaces they are offering, and one for Seekers to manage their saved spaces and active conversations.

User prompt: First, create a way for a new user to access the registration page from the homepage and the login page

Response: Here are the updates for `homepage.jsx` and `login.jsx` to let users navigate seamlessly to the registration page.

1. Update Navigation in `homepage.jsx`

Import `Link` from `react-router-dom` and update the navigation bar so non-authenticated users see options for both **Sign In** and **Get Started** (Register). A direct call-to-action button is also added inside the Hero section.

Update the top of `homepage.jsx`:

```
import React, { useEffect } from 'react';
import { useDispatch, useSelector } from 'react-redux';
import { Link } from 'react-router-dom'; // Added Link import
import { fetchListings } from '../redux/listingsSlice';
import { useAuthContext } from '../context/AuthContext';
import './homepage.css';
```

Update the `<header>` nav links section inside `homepage.jsx`:

```
<ul className="nav-links">
    <li><a href="#features">Our Mission</a></li>
    <li><a href="#listings">Find Safe Spaces</a></li>
    {userToken ? (
        <li><Link to="/dashboard" className="btn-primary">Dashboard</Link></li>
    ) : (
        <>
            <li><Link to="/login">Sign In</Link></li>
            <li><Link to="/register" className="btn-primary">Get Started</Link></li>
        </>
    )}
</ul>
```

(Optional Hero CTA) Add a registration button inside the hero content right under your subtitle:

```
<div className="hero-content">
    <h1>Find a <span>Safe Place</span> to Land</h1>
    <p>Our goal is to match you with a space where you will actually thrive, and get you out of a nest where you can

    {!userToken && (
        <div style={{ marginBottom: '2rem' }}>
            <Link to="/register" className="btn-primary" style={{ padding: '0.85rem 1.75rem', fontSize: '1rem', text
                Create an Account to Connect
            </Link>
        </div>
    )}

    {/* Search container stays here... */}
</div>
```

2. Update `login.jsx` to include Registration Link

Ensure your `login.jsx` component imports `Link` and includes a callout at the bottom directing unregistered users to the signup page.

frontend/src/components/login.jsx

```
import React, { useState } from 'react';
import axios from 'axios';
import { useNavigate, Link } from 'react-router-dom';
import { useAuthContext } from '../context/AuthContext';

const Login = () => {
    const [email, setEmail] = useState('');
    const [password, setPassword] = useState('');
    const [error, setError] = useState('');
    const [isLoading, setIsLoading] = useState(false);
    const { loginUser } = useAuthContext();
    const navigate = useNavigate();

    const handleLogin = async (e) => {
        e.preventDefault();
        if (!email || !password) return setError('Please enter both email and password');

        setIsLoading(true);
        setError('');

        try {
            const response = await axios.post('http://localhost:5000/api/auth/login', { email, password });
            loginUser(response.data.token, response.data.user);
            navigate('/dashboard');
        } catch (err) {
            setError(err.response?.data?.message || 'Invalid email or password');
        } finally {
            setIsLoading(false);
        }
    };

    return (
        <div style={{ maxWidth: '420px', margin: '4rem auto', padding: '2.5rem', border: '1px solid #e2e8f0', border
            <h2 style={{ marginBottom: '0.5rem', color: '#0f172a' }}>Welcome Back</h2>
            <p style={{ color: '#64748b', marginBottom: '2rem', fontSize: '0.95rem' }}>Sign in to manage your space

            {error && <div style={{ backgroundColor: '#fee2e2', color: '#ef4444', padding: '0.75rem', borderRadius:

            <form onSubmit={handleLogin}>
                <div style={{ marginBottom: '1.25rem' }}>
                    <label style={{ display: 'block', marginBottom: '0.5rem', fontWeight: '500', fontSize: '0.9rem'
                    <input type="email" value={email} onChange={(e) => setEmail(e.target.value)} required style={{ w
                </div>
                <div style={{ marginBottom: '1.5rem' }}>
```

```
                    <label style={{ display: 'block', marginBottom: '0.5rem', fontWeight: '500', fontSize: '0.9rem'
                    <input type="password" value={password} onChange={(e) => setPassword(e.target.value)} required s
                </div>
                <button type="submit" disabled={isLoading} style={{ width: '100%', padding: '0.85rem', background: '
                    {isLoading ? 'Signing In...' : 'Sign In'}
                </button>
            </form>

            <p style={{ marginTop: '1.5rem', textAlign: 'center', fontSize: '0.9rem', color: '#64748b' }}>
                Don't have an account yet? <Link to="/register" style={{ color: '#2563eb', textDecoration: 'none', f
            </p>
        </div>
    );
};

export default Login;
```

User prompt: Now, make the dashboard according to your suggestion

Response: Here is the implementation for the role-specific **`Dashboard.jsx`**. It dynamically renders a **Host Dashboard** (to post and manage safe spaces) or a **Seeker Dashboard** (with active requests, saved spaces, and a safety quick-exit feature).

`frontend/src/components/Dashboard.jsx`

```
import React, { useState, useEffect } from 'react';
import axios from 'axios';
import { useAuthContext } from '../context/AuthContext';
import { useNavigate } from 'react-router-dom';

const Dashboard = () => {
    const { user, userToken, logoutUser } = useAuthContext();
    const navigate = useNavigate();

    // Host Form State
    const [formData, setFormData] = useState({
        title: '',
        location: '',
        spaceType: 'Spare Room',
        price: 0,
        beds: 1,
        baths: 1,
        maxDuration: 14,
        rules: 'Quiet hours after 10 PM, pet friendly'
    });

    const [myListings, setMyListings] = useState([]);
    const [loading, setLoading] = useState(false);
    const [message, setMessage] = useState('');

    // Fetch user listings
    useEffect(() => {
        fetchListings();
    }, []);

    const fetchListings = async () => {
        try {
            const res = await axios.get('http://localhost:5000/api/listings');
            // Filter listings owned by this user (or show all for demo)
            if (user?.id) {
                setMyListings(res.data.filter(item => item.user === user.id || item.user?._id === user.id));
            } else {
                setMyListings(res.data);
            }
        } catch (err) {
            console.error('Failed to fetch listings', err);
        }
    };

    const handleFormSubmit = async (e) => {
        e.preventDefault();
        setLoading(true);
        setMessage('');

        try {
            await axios.post(
                'http://localhost:5000/api/listings',
                formData,
                { headers: { Authorization: `Bearer ${userToken}` } }
```

```
            );
            setMessage('Space posted successfully!');
            setFormData({
                title: '',
                location: '',
                spaceType: 'Spare Room',
                price: 0,
                beds: 1,
                baths: 1,
                maxDuration: 14,
                rules: ''
            });
            fetchListings();
        } catch (err) {
            setMessage(err.response?.data?.message || 'Failed to post space');
        } finally {
            setLoading(false);
        }
    };

    const handleDelete = async (id) => {
        if (!window.confirm('Are you sure you want to remove this space?')) return;
        try {
            await axios.delete(`http://localhost:5000/api/listings/${id}`, {
                headers: { Authorization: `Bearer ${userToken}` }
            });
            fetchListings();
        } catch (err) {
            alert('Failed to delete listing');
        }
    };

    // Emergency Quick Exit for Seekers
    const quickExit = () => {
        window.location.href = 'https://www.google.com';
    };

    return (
        <div style={{ maxWidth: '1100px', margin: '2rem auto', padding: '0 1.5rem' }}>
            {/* Top Navigation Bar */}
            <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', paddingBottom: '1.
                <div>
                    <h1 style={{ fontSize: '1.75rem', color: '#0f172a', margin: 0 }}>
                        {user?.role === 'host' ? 'Host Community Portal' : 'Seeker Support Portal'}
                    </h1>
                    <p style={{ color: '#64748b', margin: '0.25rem 0 0 0', fontSize: '0.95rem' }}>
                        Logged in as <strong>{user?.email}</strong> ({user?.role?.toUpperCase() || 'MEMBER'})
                    </p>
                </div>
                <div style={{ display: 'flex', gap: '1rem', alignItems: 'center' }}>
                    {user?.role === 'seeker' && (
                        <button
                            onClick={quickExit}
                            style={{ backgroundColor: '#dc2626', color: '#fff', border: 'none', padding: '0.6rem 1re
                            <i className="fa-solid fa-person-running"></i> Quick Exit
                        </button>
                    )}
                    <button
                        onClick={() => { logoutUser(); navigate('/'); }}
                        style={{ backgroundColor: 'transparent', border: '1px solid #cbd5e1', padding: '0.6rem 1rem'
                        Sign Out
                    </button>
                </div>
            </div>

            {/* HOST DASHBOARD VIEW */}
            {user?.role === 'host' ? (
                <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '2rem', marginTop: '2rem' }}>
                    {/* Post Space Form */}
                    <div style={{ backgroundColor: '#ffffff', padding: '1.75rem', borderRadius: '12px', border: '1px
                        <h2 style={{ fontSize: '1.25rem', marginBottom: '1rem', color: '#0f172a' }}>Offer a Safe Spa
                        {message && <p style={{ padding: '0.5rem', borderRadius: '4px', backgroundColor: message.inc

                        <form onSubmit={handleFormSubmit} style={{ display: 'flex', flexDirection: 'column', gap: '1
                            <div>
                                <label style={{ display: 'block', fontSize: '0.85rem', fontWeight: '600', marginBott
                                <input type="text" placeholder="e.g. Peaceful Private Bedroom" value={formData.title
                            </div>
                            <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '1rem' }}>
```

```
                        <div>
                            <label style={{ display: 'block', fontSize: '0.85rem', fontWeight: '600', margin
                            <input type="text" placeholder="e.g. Austin, TX" value={formData.location} onCha
                        </div>
                        <div>
                            <label style={{ display: 'block', fontSize: '0.85rem', fontWeight: '600', margin
                            <select value={formData.spaceType} onChange={(e) => setFormData({...formData, sp
                                <option value="Spare Room">Spare Room</option>
                                <option value="Living Room Couch">Living Room Couch</option>
                                <option value="In-law Suite">In-law Suite</option>
                                <option value="Entire Apartment">Entire Apartment</option>
                            </select>
                        </div>
                    </div>
                    <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr 1fr', gap: '0.75rem' }}>
                        <div>
                            <label style={{ display: 'block', fontSize: '0.85rem', fontWeight: '600', margin
                            <input type="number" min="0" value={formData.price} onChange={(e) => setFormData
                        </div>
                        <div>
                            <label style={{ display: 'block', fontSize: '0.85rem', fontWeight: '600', margin
                            <input type="number" min="1" value={formData.maxDuration} onChange={(e) => setFo
                        </div>
                        <div>
                            <label style={{ display: 'block', fontSize: '0.85rem', fontWeight: '600', margin
                            <input type="number" min="1" value={formData.beds} onChange={(e) => setFormData(
                        </div>
                    </div>
                    <div>
                        <label style={{ display: 'block', fontSize: '0.85rem', fontWeight: '600', marginBott
                        <textarea rows="3" value={formData.rules} onChange={(e) => setFormData({...formData,
                    </div>
                    <button type="submit" disabled={loading} style={{ padding: '0.75rem', backgroundColor: '
                        {loading ? 'Publishing...' : 'Publish Space Offer'}
                    </button>
                </form>
            </div>

            {/* Manage Posted Spaces */}
            <div>
                <h2 style={{ fontSize: '1.25rem', marginBottom: '1rem', color: '#0f172a' }}>Your Offended Sp
                {myListings.length === 0 ? (
                    <p style={{ color: '#64748b' }}>You haven't posted any spaces yet.</p>
                ) : (
                    <div style={{ display: 'flex', flexDirection: 'column', gap: '1rem' }}>
                        {myListings.map((item) => (
                            <div key={item._id} style={{ backgroundColor: '#ffffff', padding: '1.25rem', bor
                                <div>
                                    <h3 style={{ margin: 0, fontSize: '1rem', color: '#0f172a' }}>{item.titl
                                    <p style={{ margin: '0.25rem 0', color: '#64748b', fontSize: '0.85rem' }
                                    <span style={{ fontSize: '0.75rem', backgroundColor: '#f1f5f9', padding:
                                </div>
                                <button onClick={() => handleDelete(item._id)} style={{ backgroundColor: '#f
                                    Remove
                                </button>
                            </div>
                        ))}
                    </div>
                )}
            </div>
        </div>
    ) : (
        /* SEEKER DASHBOARD VIEW */
        <div style={{ marginTop: '2rem' }}>
            <div style={{ backgroundColor: '#f0f9ff', border: '1px solid #bae6fd', padding: '1.5rem', border
                <h2 style={{ color: '#0369a1', marginTop: 0, fontSize: '1.25rem' }}>Safety & Privacy Checkli
                <ul style={{ color: '#0c4a6e', paddingLeft: '1.25rem', margin: '0.5rem 0 0 0', lineHeight: '
                    <li>Never share your exact current location until you feel comfortable with a verified h
                    <li>Use our internal messaging to communicate before meeting.</li>
                    <li>Use the red <strong>Quick Exit</strong> button at the top right to immediately leave
                </ul>
            </div>

            <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', marginBott
                <h2 style={{ fontSize: '1.25rem', color: '#0f172a', margin: 0 }}>Explore Safe Spaces Nearby<
                <button onClick={() => navigate('/')} style={{ backgroundColor: '#2563eb', color: '#white',
                    Browse All Postings
                </button>
            </div>
```

```
                    <div style={{ display: 'grid', gridTemplateColumns: 'repeat(auto-fill, minmax(300px, 1fr))', gap
                        {myListings.slice(0, 6).map((listing) => (
                            <div key={listing._id} style={{ backgroundColor: '#ffffff', borderRadius: '10px', paddin
                                <span style={{ backgroundColor: '#dcfce7', color: '#166534', padding: '0.2rem 0.5rem
                                    {listing.price === 0 ? 'Free Accommodation' : `$${listing.price}/mo`}
                                </span>
                                <h3 style={{ fontSize: '1.1rem', margin: '0.75rem 0 0.25rem 0' }}>{listing.title}</h
                                <p style={{ color: '#64748b', fontSize: '0.85rem', margin: 0 }}>📍 {listing.locatior
                                <div style={{ margin: '1rem 0', fontSize: '0.85rem', color: '#334155' }}>
                                    <p style={{ margin: '0.25rem 0' }}>⏱️ Max Stay: <strong>{listing.maxDuration ||
                                    <p style={{ margin: '0.25rem 0' }}>🏡 Space: <strong>{listing.spaceType || 'Spar
                                </div>
                                <button style={{ width: '100%', padding: '0.6rem', backgroundColor: '#f1f5f9', color
                                    Request Support
                                </button>
                            </div>
                        ))}
                    </div>
                </div>
            )}
        </div>
    );
};

export default Dashboard;
```

Integration Step Ensure your `frontend/src/App.jsx` imports `Dashboard.jsx` correctly:

```
import Dashboard from './components/Dashboard';

// Inside <Routes>:
<Route
  path="/dashboard"
  element={
    <ProtectedRoute>
      <Dashboard />
    </ProtectedRoute>
  }
/>
```

User prompt: Let's add a log out button on the homepage and dashboard

Response: Here are the updated nav/header code snippets for both `homepage.jsx` and `Dashboard.jsx` to handle logging out.

1. Update `homepage.jsx`

Destructure `logoutUser` from `useAuthContext` and add the Log Out button directly into the conditionally rendered navigation links.

```
// Near the top of homepage.jsx:
const { userToken, logoutUser } = useAuthContext();

// Inside the <header> <nav> <ul className="nav-links">:
<ul className="nav-links">
    <li><a href="#features">Our Mission</a></li>
    <li><a href="#listings">Find Safe Spaces</a></li>
    {userToken ? (
        <>
            <li><Link to="/dashboard" className="btn-primary">Dashboard</Link></li>
            <li>
                <button
                    onClick={logoutUser}
                    style={{
                        background: 'transparent',
                        border: '1px solid #cbd5e1',
                        color: '#475569',
                        padding: '0.6rem 1.1rem',
                        borderRadius: '8px',
                        cursor: 'pointer',
                        fontWeight: '500',
                        fontSize: '0.95rem',
                        transition: 'all 0.2s ease'
                    }}
                >
                    Log Out
                </button>
            </li>
        </>
```

```
    ) : (
        <>
            <li><Link to="/login">Sign In</Link></li>
            <li><Link to="/register" className="btn-primary">Get Started</Link></li>
        </>
    )}
</ul>
```

2. Update `Dashboard.jsx` Header

Ensure the Sign Out button in `Dashboard.jsx` calls `logoutUser()` and redirects back to the homepage.

```
// Inside the Top Navigation Bar of Dashboard.jsx:
<div style={{ display: 'flex', gap: '1rem', alignItems: 'center' }}>
    {user?.role === 'seeker' && (
        <button
            onClick={quickExit}
            style={{
                backgroundColor: '#dc2626',
                color: '#ffffff',
                border: 'none',
                padding: '0.6rem 1rem',
                borderRadius: '6px',
                cursor: 'pointer',
                fontWeight: 'bold'
            }}
        >
            <i className="fa-solid fa-person-running"></i> Quick Exit
        </button>
    )}
    <button
        onClick={() => {
            logoutUser();
            navigate('/');
        }}
        style={{
            backgroundColor: 'transparent',
            border: '1px solid #cbd5e1',
            color: '#475569',
            padding: '0.6rem 1rem',
            borderRadius: '6px',
            cursor: 'pointer',
            fontWeight: '500'
        }}
    >
        Log Out
    </button>
</div>
```

User prompt: The existing registration page doesn't allow a user to choose a role (the attempt to create a new user contained an email which I deleted for security purposes)

Response: Your React code is correctly sending the `role` field in `axios.post()`, but your backend `authController.js` is not extracting `role` from `req.body` or passing it into `User.create()`. Because Mongoose requires a `role` field in your User schema, it throws `User validation failed: role: Path 'role' is required.`

Update your backend registration logic to accept and save `role`:

backend/controllers/authController.js

```
const User = require('../models/User');
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');

exports.register = async (req, res) => {
    try {
        // 1. Destructure role from the request body
        const { email, password, role } = req.body;

        // 2. Validate role presence fallback
        const userRole = role || 'seeker';

        // 3. Check for existing user
        const existingUser = await User.findOne({ email });
        if (existingUser) {
            return res.status(400).json({ message: 'User already exists' });
        }
```

```
        // 4. Hash password
        const hashedPassword = await bcrypt.hash(password, 10);

        // 5. Pass userRole into User.create
        const user = await User.create({
            email,
            password: hashedPassword,
            role: userRole
        });

        // 6. Sign JWT payload including role
        const token = jwt.sign(
            { id: user._id, role: user.role },
            process.env.JWT_SECRET || 'secretkey',
            { expiresIn: '7d' }
        );

        res.status(201).json({
            token,
            user: {
                id: user._id,
                email: user.email,
                role: user.role
            }
        });
    } catch (error) {
        res.status(500).json({ message: error.message });
    }
};
```

If you are using **Joi** for request body validation inside the controller, ensure your schema includes the field:

```
const registerSchema = Joi.object({
    email: Joi.string().email().required(),
    password: Joi.string().min(6).required(),
    role: Joi.string().valid('seeker', 'host').required()
});
```

User prompt: Help me add an option to upload pictures as a host

Response: To handle image uploads seamlessly without needing external storage services or complex backend file routing, the simplest approach is to convert uploaded images into a **Base64 string** on the frontend and send it directly in your JSON request. This saves the image data directly to your existing `image` field in MongoDB.

Here is how to update both your frontend host form and backend controller.

1. Update Frontend (`frontend/src/components/Dashboard.jsx`)

Add `image: ''` to your `formData` state, create a file reader handler, and add the file input field with a preview in the Host form.

```
// 1. In Dashboard.jsx, update your formData initial state:
const [formData, setFormData] = useState({
    title: '',
    location: '',
    spaceType: 'Spare Room',
    price: 0,
    beds: 1,
    baths: 1,
    maxDuration: 14,
    rules: 'Quiet hours after 10 PM, pet friendly',
    image: '' // Added image field
});

// 2. Add this image conversion handler inside your Dashboard component:
const handleImageUpload = (e) => {
    const file = e.target.files[0];
    if (file) {
        // Limit file size to ~2MB for database efficiency
        if (file.size > 2 * 1024 * 1024) {
            alert('File size is too large. Please select an image under 2MB.');
            return;
        }
        const reader = new FileReader();
        reader.onloadend = () => {
            setFormData(prev => ({ ...prev, image: reader.result }));
        };
        reader.readAsDataURL(file);
    }
```

```
};

// 3. Reset image in handleFormSubmit after successful post:
setFormData({
    title: '',
    location: '',
    spaceType: 'Spare Room',
    price: 0,
    beds: 1,
    baths: 1,
    maxDuration: 14,
    rules: '',
    image: ''
});
```

Add this Input to the Host Form JSX (right above the submit button):

```
<div>
    <label style={{ display: 'block', fontSize: '0.85rem', fontWeight: '600', marginBottom: '0.25rem' }}>
        Upload Space Photo
    </label>
    <input
        type="file"
        accept="image/*"
        onChange={handleImageUpload}
        style={{ width: '100%', padding: '0.5rem', borderRadius: '6px', border: '1px solid #cbd5e1', backgroundColor
    />
    {formData.image && (
        <div style={{ marginTop: '0.5rem' }}>
            <img
                src={formData.image}
                alt="Preview"
                style={{ width: '100%', height: '140px', objectFit: 'cover', borderRadius: '6px' }}
            />
        </div>
    )}
</div>
```

2. Update Backend Controller (`backend/controllers/listingController.js`)

Ensure your `createListing` function extracts the `image` property from `req.body` and stores it when calling `Listing.create()`.

```
exports.createListing = async (req, res) => {
    try {
        const { title, location, spaceType, price, beds, baths, maxDuration, rules, image } = req.body;

        const newListing = await Listing.create({
            title,
            location,
            spaceType,
            price,
            beds,
            baths,
            maxDuration,
            rules,
            image: image || 'https://images.unsplash.com/photo-1522708323590-d24dbb6b0267?auto=format&fit=crop&w=500
            user: req.user.id
        });

        res.status(201).json(newListing);
    } catch (error) {
        res.status(500).json({ message: error.message });
    }
};
```

User prompt: After implementing the changes, the dashboard went blank. How do I fix?

Response: The blank screen is caused by an infinite re-render loop: `setFormData({...})` was placed directly inside the main body of the component instead of inside `handleFormSubmit`. Executing state updates during render crashes React instantly.

Replace the contents of **`Dashboard.jsx`** with this corrected code:

```
import React, { useState, useEffect } from 'react';
import axios from 'axios';
import { useAuthContext } from '../context/AuthContext';
import { useNavigate } from 'react-router-dom';

const Dashboard = () => {
```

```
        const { user, userToken, logoutUser } = useAuthContext();
        const navigate = useNavigate();

        // Host Form State
        const [formData, setFormData] = useState({
            title: '',
            location: '',
            spaceType: 'Spare Room',
            price: 0,
            beds: 1,
            baths: 1,
            maxDuration: 14,
            rules: 'Quiet hours after 10 PM, pet friendly',
            image: ''
        });

        const [myListings, setMyListings] = useState([]);
        const [loading, setLoading] = useState(false);
        const [message, setMessage] = useState('');

        // Fetch user listings
        useEffect(() => {
            fetchListings();
        }, []);

        const fetchListings = async () => {
            try {
                const res = await axios.get('http://localhost:5000/api/listings');
                if (user?.id) {
                    setMyListings(res.data.filter(item => item.user === user.id || item.user?._id === user.id));
                } else {
                    setMyListings(res.data);
                }
            } catch (err) {
                console.error('Failed to fetch listings', err);
            }
        };

        const handleFormSubmit = async (e) => {
            e.preventDefault();
            setLoading(true);
            setMessage('');

            try {
                await axios.post(
                    'http://localhost:5000/api/listings',
                    formData,
                    { headers: { Authorization: `Bearer ${userToken}` } }
                );
                setMessage('Space posted successfully!');

                // Reset form after successful submission
                setFormData({
                    title: '',
                    location: '',
                    spaceType: 'Spare Room',
                    price: 0,
                    beds: 1,
                    baths: 1,
                    maxDuration: 14,
                    rules: '',
                    image: ''
                });

                fetchListings();
            } catch (err) {
                setMessage(err.response?.data?.message || 'Failed to post space');
            } finally {
                setLoading(false);
            }
        };

        const handleDelete = async (id) => {
            if (!window.confirm('Are you sure you want to remove this space?')) return;
            try {
                await axios.delete(`http://localhost:5000/api/listings/${id}`, {
                    headers: { Authorization: `Bearer ${userToken}` }
                });
                fetchListings();
```

```
        } catch (err) {
            alert('Failed to delete listing');
        }
    };

    const handleImageUpload = (e) => {
        const file = e.target.files[0];
        if (file) {
            if (file.size > 2 * 1024 * 1024) {
                alert('File size is too large. Please select an image under 2MB.');
                return;
            }
            const reader = new FileReader();
            reader.onloadend = () => {
                setFormData(prev => ({ ...prev, image: reader.result }));
            };
            reader.readAsDataURL(file);
        }
    };

    const quickExit = () => {
        window.location.href = 'https://www.google.com';
    };

    return (
        <div style={{ maxWidth: '1100px', margin: '2rem auto', padding: '0 1.5rem' }}>
            {/* Top Navigation Bar */}
            <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', paddingBottom: '1.
                <div>
                    <h1 style={{ fontSize: '1.75rem', color: '#0f172a', margin: 0 }}>
                        {user?.role === 'host' ? 'Host Community Portal' : 'Seeker Support Portal'}
                    </h1>
                    <p style={{ color: '#64748b', margin: '0.25rem 0 0 0', fontSize: '0.95rem' }}>
                        Logged in as <strong>{user?.email}</strong> ({user?.role?.toUpperCase() || 'MEMBER'})
                    </p>
                </div>
                <div style={{ display: 'flex', gap: '1rem', alignItems: 'center' }}>
                    {user?.role === 'seeker' && (
                        <button
                            onClick={quickExit}
                            style={{ backgroundColor: '#dc2626', color: '#fff', border: 'none', padding: '0.6rem 1re
                            <i className="fa-solid fa-person-running"></i> Quick Exit
                        </button>
                    )}
                    <button
                        onClick={() => { logoutUser(); navigate('/'); }}
                        style={{ backgroundColor: 'transparent', border: '1px solid #cbd5e1', padding: '0.6rem 1rem'
                        Sign Out
                    </button>
                </div>
            </div>

            {/* HOST DASHBOARD VIEW */}
            {user?.role === 'host' ? (
                <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '2rem', marginTop: '2rem' }}>
                    {/* Post Space Form */}
                    <div style={{ backgroundColor: '#ffffff', padding: '1.75rem', borderRadius: '12px', border: '1px
                        <h2 style={{ fontSize: '1.25rem', marginBottom: '1rem', color: '#0f172a' }}>Offer a Safe Spa
                        {message && <p style={{ padding: '0.5rem', borderRadius: '4px', backgroundColor: message.inc

                        <form onSubmit={handleFormSubmit} style={{ display: 'flex', flexDirection: 'column', gap: '1
                            <div>
                                <label style={{ display: 'block', fontSize: '0.85rem', fontWeight: '600', marginBott
                                <input type="text" placeholder="e.g. Peaceful Private Bedroom" value={formData.title
                            </div>
                            <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '1rem' }}>
                                <div>
                                    <label style={{ display: 'block', fontSize: '0.85rem', fontWeight: '600', margin
                                    <input type="text" placeholder="e.g. Tel Aviv" value={formData.location} onChang
                                </div>
                                <div>
                                    <label style={{ display: 'block', fontSize: '0.85rem', fontWeight: '600', margin
                                    <select value={formData.spaceType} onChange={(e) => setFormData({...formData, sp
                                        <option value="Spare Room">Spare Room</option>
                                        <option value="Living Room Couch">Living Room Couch</option>
                                        <option value="In-law Suite">In-law Suite</option>
                                        <option value="Entire Apartment">Entire Apartment</option>
                                    </select>
                                </div>
```

```
                    </div>
                    <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr 1fr', gap: '0.75rem' }}>
                        <div>
                            <label style={{ display: 'block', fontSize: '0.85rem', fontWeight: '600', margin
                            <input type="number" min="0" value={formData.price} onChange={(e) => setFormData
                        </div>
                        <div>
                            <label style={{ display: 'block', fontSize: '0.85rem', fontWeight: '600', margin
                            <input type="number" min="1" value={formData.maxDuration} onChange={(e) => setFo
                        </div>
                        <div>
                            <label style={{ display: 'block', fontSize: '0.85rem', fontWeight: '600', margin
                            <input type="number" min="1" value={formData.beds} onChange={(e) => setFormData(
                        </div>
                    </div>
                    <div>
                        <label style={{ display: 'block', fontSize: '0.85rem', fontWeight: '600', marginBott
                        <textarea rows="3" value={formData.rules} onChange={(e) => setFormData({...formData,
                    </div>
                    <div>
                        <label style={{ display: 'block', fontSize: '0.85rem', fontWeight: '600', marginBott
                        <input
                            type="file"
                            accept="image/*"
                            onChange={handleImageUpload}
                            style={{ width: '100%', padding: '0.5rem', borderRadius: '6px', border: '1px sol
                        />
                        {formData.image && (
                            <div style={{ marginTop: '0.5rem' }}>
                                <img
                                    src={formData.image}
                                    alt="Preview"
                                    style={{ width: '100%', height: '140px', objectFit: 'cover', borderRadiu
                                />
                            </div>
                        )}
                    </div>
                    <button type="submit" disabled={loading} style={{ padding: '0.75rem', backgroundColor: '
                        {loading ? 'Publishing...' : 'Publish Space Offer'}
                    </button>
                </form>
            </div>

            {/* Manage Posted Spaces */}
            <div>
                <h2 style={{ fontSize: '1.25rem', marginBottom: '1rem', color: '#0f172a' }}>Your Offered Spa
                {myListings.length === 0 ? (
                    <p style={{ color: '#64748b' }}>You haven't posted any spaces yet.</p>
                ) : (
                    <div style={{ display: 'flex', flexDirection: 'column', gap: '1rem' }}>
                        {myListings.map((item) => (
                            <div key={item._id} style={{ backgroundColor: '#ffffff', padding: '1.25rem', bor
                                <div>
                                    <h3 style={{ margin: 0, fontSize: '1rem', color: '#0f172a' }}>{item.titl
                                    <p style={{ margin: '0.25rem 0', color: '#64748b', fontSize: '0.85rem' }
                                    <span style={{ fontSize: '0.75rem', backgroundColor: '#f1f5f9', padding:
                                </div>
                                <button onClick={() => handleDelete(item._id)} style={{ backgroundColor: '#f
                                    Remove
                                </button>
                            </div>
                        ))}
                    </div>
                )}
            </div>
        </div>
    ) : (
        /* SEEKER DASHBOARD VIEW */
        <div style={{ marginTop: '2rem' }}>
            <div style={{ backgroundColor: '#f0f9ff', border: '1px solid #bae6fd', padding: '1.5rem', border
                <h2 style={{ color: '#0369a1', marginTop: 0, fontSize: '1.25rem' }}>Safety & Privacy Checkli
                <ul style={{ color: '#0c4a6e', paddingLeft: '1.25rem', margin: '0.5rem 0 0 0', lineHeight: '
                    <li>Never share your exact current location until you feel comfortable with a verified h
                    <li>Use our internal messaging to communicate before meeting.</li>
                    <li>Use the red <strong>Quick Exit</strong> button at the top right to immediately leave
                </ul>
            </div>

            <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', marginBott
```

```
                        <h2 style={{ fontSize: '1.25rem', color: '#0f172a', margin: 0 }}>Explore Safe Spaces Nearby<
                        <button onClick={() => navigate('/')} style={{ backgroundColor: '#2563eb', color: '#white',
                            Browse All Postings
                        </button>
                    </div>

                    <div style={{ display: 'grid', gridTemplateColumns: 'repeat(auto-fill, minmax(300px, 1fr))', gap
                        {myListings.slice(0, 6).map((listing) => (
                            <div key={listing._id} style={{ backgroundColor: '#ffffff', borderRadius: '10px', paddin
                                <span style={{ backgroundColor: '#dcfce7', color: '#166534', padding: '0.2rem 0.5rem
                                    {listing.price === 0 ? 'Free Accommodation' : `$${listing.price}/mo`}
                                </span>
                                <h3 style={{ fontSize: '1.1rem', margin: '0.75rem 0 0.25rem 0' }}>{listing.title}</h
                                <p style={{ color: '#64748b', fontSize: '0.85rem', margin: 0 }}>📍 {listing.locatior
                                <div style={{ margin: '1rem 0', fontSize: '0.85rem', color: '#334155' }}>
                                    <p style={{ margin: '0.25rem 0' }}>⏱️ Max Stay: <strong>{listing.maxDuration ||
                                    <p style={{ margin: '0.25rem 0' }}>🏠 Space: <strong>{listing.spaceType || 'Spar
                                </div>
                                <button style={{ width: '100%', padding: '0.6rem', backgroundColor: '#f1f5f9', color
                                    Request Support
                                </button>
                            </div>
                        ))}
                    </div>
                </div>
            )}
        </div>
    );
};

export default Dashboard;
```

User prompt: Help me add a role slot in the login page

Response: Here is the updated **`login.jsx`** component with a role selection dropdown, along with the optional backend check for **`authController.js`** to verify that the logging-in user matches the selected role.

1. Update Frontend (`login.jsx`)

Add the `role` state and a select dropdown right above the email input field.

```
import React, { useState } from 'react';
import axios from 'axios';
import { useNavigate, Link } from 'react-router-dom';
import { useAuthContext } from '../context/AuthContext';

const Login = () => {
    const [email, setEmail] = useState('');
    const [password, setPassword] = useState('');
    const [role, setRole] = useState('seeker'); // Added role state
    const [error, setError] = useState('');
    const [isLoading, setIsLoading] = useState(false);
    const { loginUser } = useAuthContext();
    const navigate = useNavigate();

    const handleLogin = async (e) => {
        e.preventDefault();
        if (!email || !password) return setError('Please enter both email and password');

        setIsLoading(true);
        setError('');

        try {
            // Pass role along with email and password
            const response = await axios.post('http://localhost:5000/api/auth/login', { email, password, role });
            loginUser(response.data.token, response.data.user);
            navigate('/dashboard');
        } catch (err) {
            setError(err.response?.data?.message || 'Invalid login credentials');
        } finally {
            setIsLoading(false);
        }
    };

    return (
        <div style={{ maxWidth: '420px', margin: '4rem auto', padding: '2.5rem', border: '1px solid #e2e8f0', border
            <h2 style={{ marginBottom: '0.5rem', color: '#0f172a' }}>Welcome Back</h2>
```

```
            <p style={{ color: '#64748b', marginBottom: '2rem', fontSize: '0.95rem' }}>Sign in to manage your space

            {error && <div style={{ backgroundColor: '#fee2e2', color: '#ef4444', padding: '0.75rem', borderRadius:

            <form onSubmit={handleLogin}>
                {/* Role Selector Slot */}
                <div style={{ marginBottom: '1.25rem' }}>
                    <label style={{ display: 'block', marginBottom: '0.5rem', fontWeight: '500', fontSize: '0.9rem'
                    <select value={role} onChange={(e) => setRole(e.target.value)} style={{ width: '100%', padding:
                        <option value="seeker">Seeker (Looking for a space)</option>
                        <option value="host">Host (Offering a space)</option>
                    </select>
                </div>

                <div style={{ marginBottom: '1.25rem' }}>
                    <label style={{ display: 'block', marginBottom: '0.5rem', fontWeight: '500', fontSize: '0.9rem'
                    <input type="email" value={email} onChange={(e) => setEmail(e.target.value)} required style={{ w
                </div>

                <div style={{ marginBottom: '1.5rem' }}>
                    <label style={{ display: 'block', marginBottom: '0.5rem', fontWeight: '500', fontSize: '0.9rem'
                    <input type="password" value={password} onChange={(e) => setPassword(e.target.value)} required s
                </div>

                <button type="submit" disabled={isLoading} style={{ width: '100%', padding: '0.85rem', background: '
                    {isLoading ? 'Signing In...' : 'Sign In'}
                </button>
            </form>

            <p style={{ marginTop: '1.5rem', textAlign: 'center', fontSize: '0.9rem', color: '#64748b' }}>
                Don't have an account yet? <Link to="/register" style={{ color: '#2563eb', textDecoration: 'none', f
            </p>
        </div>
    );
};

export default Login;
```

2. Update Backend (`backend/controllers/authController.js`)

If you want the backend to reject logins where the selected role doesn't match the account's registered role in the database, update your `login` controller function:

```
exports.login = async (req, res) => {
    try {
        const { email, password, role } = req.body;

        // 1. Find user by email
        const user = await User.findOne({ email });
        if (!user) {
            return res.status(400).json({ message: 'Invalid credentials' });
        }

        // 2. Validate role match (optional security check)
        if (role && user.role !== role) {
            return res.status(400).json({
                message: `This email is registered as a ${user.role}. Please select the correct role option.`
            });
        }

        // 3. Verify password
        const isMatch = await bcrypt.compare(password, user.password);
        if (!isMatch) {
            return res.status(400).json({ message: 'Invalid credentials' });
        }

        // 4. Return JWT Token
        const token = jwt.sign(
            { id: user._id, role: user.role },
            process.env.JWT_SECRET || 'secretkey',
            { expiresIn: '7d' }
        );

        res.json({
            token,
            user: {
                id: user._id,
                email: user.email,
                role: user.role
```

```
            }
        });
    } catch (error) {
        res.status(500).json({ message: error.message });
    }
};
```

User prompt: What's going on here, how to fix?

Response: The `request entity too large` error occurs because Express defaults to a 100 KB payload limit for JSON requests, which is easily exceeded when sending Base64-encoded image strings.

Update your backend entry file (`server.js` or `app.js`) to increase the request size limit:

```
// Replace app.use(express.json()); with:
app.use(express.json({ limit: '10mb' }));
app.use(express.urlencoded({ limit: '10mb', extended: true }));
```

Restart your backend server after saving this change, and the image upload will process without hitting the limit.

User prompt: Where exatcly in server.js do I add the line?

Response: Replace line 13 (`app.use(express.json());`) directly below `app.use(cors());`.

Here is your updated `server.js` file:

```
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
const path = require('path');
require('dotenv').config();

const authRoutes = require('./routes/authRoutes');
const listingRoutes = require('./routes/listingRoutes');
const errorHandler = require('./middlewares/errorMiddleware');

const app = express();

app.use(cors());

// Updated payload limits for image upload handling
app.use(express.json({ limit: '10mb' }));
app.use(express.urlencoded({ limit: '10mb', extended: true }));

app.use('/uploads', express.static(path.join(__dirname, 'uploads')));

app.use('/api/auth', authRoutes);
app.use('/api/listings', listingRoutes);

app.use(errorHandler);

const PORT = process.env.PORT || 5000;

mongoose.connect(process.env.MONGO_URI)
    .then(() => {
        console.log('MongoDB Connected');
        app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
    })
    .catch((err) => console.error('MongoDB Connection Error:', err));
```

User prompt: The search button leads nowhere. Let's change that!

Response: To help you wire up the search button, I'll need to see the code where it currently lives.

Could you share the file containing the search button (likely `homepage.jsx`), and let me know how you want the search to behave? For example, should it redirect the user to the dashboard and filter the listings there, or do you have a dedicated search results page in mind?

User prompt: How do you recommend the search to behave and why?

Response: For a platform like NestFinder where users are seeking safe spaces—often in vulnerable or time-sensitive situations—the search experience should prioritize **immediate reassurance, low friction, and host privacy**.

I recommend a **Privacy-First Public Search** approach. The search button on the homepage should redirect the user to a dedicated search results page (`/search`) rather than forcing them to log in first.

Here is exactly how this should behave and why:

- **Location-Centric Querying:** The homepage search bar should capture a city or neighborhood (e.g., "Tel Aviv") and pass it as a URL parameter (e.g., `/search?location=Tel+Aviv`).
    - *Why:* People in need primarily care about geographical proximity. Using URL parameters makes the search results shareable and allows users to refresh the page without losing their results.
- **Public Visibility with Gated Actions:** The search results page should publicly display the available listing cards (Title, General Location, Space Type, Max Days) but disable or gate the "Request Support" button.
    - *Why:* Showing actual available spaces immediately builds trust and proves the platform works, encouraging the seeker to register. However, gating the contact action ensures bad actors cannot mass-message hosts without an account, protecting your host community.
- **Empty State Empathy:** If no spaces are available in the searched area, the results page should display a compassionate message and offer an alternative, such as "No exact matches in [City], but here are spaces within 20 miles," or provide a link to emergency hotlines.
    - *Why:* A blank page or a blunt "0 results" message can be highly discouraging to someone in crisis. Providing immediate next steps keeps them supported.

How we can build this in your current stack:

1. **Homepage Update:** We capture the input value in a piece of state and use React Router's `useNavigate` to push the user to `/search?location=${searchInput}`.
2. **New Search Component (`Search.jsx`):** We build a new page that reads the URL parameter, fetches the listings from your existing `http://localhost:5000/api/listings` endpoint, and filters them on the frontend (or backend) by the requested location.
3. **Component Reuse:** We can reuse the exact same card design you already built for the seeker view in `Dashboard.jsx` so the UI remains consistent.

Would you like to build out this dedicated `Search.jsx` page, or would you prefer to route the search directly into the existing `Dashboard.jsx` and handle the filtering there?

User prompt: Build the dedicated page according to your own points

Response: Here is the complete setup for the dedicated public search page, including the `Search.jsx` component, the routing configuration, and the updated `Homepage.jsx` search form.

1. Create `frontend/src/pages/Search.jsx`

Create a new file named `Search.jsx` (or `components/Search.jsx`) to handle location queries, display matches, and guide unauthenticated users gracefully.

```
import React, { useState, useEffect } from 'react';
import axios from 'axios';
import { useSearchParams, useNavigate, Link } from 'react-router-dom';
import { useAuthContext } from '../context/AuthContext';

const Search = () => {
    const [searchParams, setSearchParams] = useSearchParams();
    const queryLocation = searchParams.get('location') || '';

    const [searchInput, setSearchInput] = useState(queryLocation);
    const [listings, setListings] = useState([]);
    const [filteredListings, setFilteredListings] = useState([]);
    const [loading, setLoading] = useState(true);

    const { userToken } = useAuthContext();
    const navigate = useNavigate();

    useEffect(() => {
        fetchListings();
    }, []);

    useEffect(() => {
        if (queryLocation.trim() === '') {
            setFilteredListings(listings);
        } else {
            const filtered = listings.filter(item =>
                item.location?.toLowerCase().includes(queryLocation.toLowerCase()) ||
                item.title?.toLowerCase().includes(queryLocation.toLowerCase())
            );
            setFilteredListings(filtered);
        }
    }, [queryLocation, listings]);

    const fetchListings = async () => {
        try {
```

```
        const res = await axios.get('http://localhost:5000/api/listings');
        setListings(res.data);
    } catch (err) {
        console.error('Failed to load listings', err);
    } finally {
        setLoading(false);
    }
};

const handleSearchSubmit = (e) => {
    e.preventDefault();
    setSearchParams({ location: searchInput });
};

const handleActionClick = () => {
    if (!userToken) {
        navigate('/register');
    } else {
        navigate('/dashboard');
    }
};

return (
    <div style={{ maxWidth: '1100px', margin: '2rem auto', padding: '0 1.5rem' }}>
        {/* Header & Search Bar */}
        <div style={{ backgroundColor: '#ffffff', padding: '2rem', borderRadius: '12px', border: '1px solid #e2e
            <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', marginBottom:
                <Link to="/" style={{ textDecoration: 'none', color: '#2563eb', fontWeight: 'bold', fontSize: '1
                    ← Back to Home
                </Link>
                {!userToken && (
                    <div style={{ fontSize: '0.9rem', color: '#64748b' }}>
                        Need to host or request support? <Link to="/login" style={{ color: '#2563eb', fontWeight
                    </div>
                )}
            </div>

            <h1 style={{ fontSize: '1.75rem', color: '#0f172a', marginBottom: '0.5rem' }}>Find Safe Accommodatio
            <p style={{ color: '#64748b', marginBottom: '1.5rem', fontSize: '0.95rem' }}>Search vetted host spac

            <form onSubmit={handleSearchSubmit} style={{ display: 'flex', gap: '0.75rem', maxWidth: '600px' }}>
                <input
                    type="text"
                    placeholder="Enter city or neighborhood (e.g. Tel Aviv)..."
                    value={searchInput}
                    onChange={(e) => setSearchInput(e.target.value)}
                    style={{ flex: 1, padding: '0.75rem 1rem', borderRadius: '8px', border: '1px solid #cbd5e1',
                />
                <button type="submit" style={{ backgroundColor: '#2563eb', color: '#ffffff', border: 'none', pad
                    Search
                </button>
            </form>
        </div>

        {/* Results Header */}
        <div style={{ marginBottom: '1.5rem' }}>
            <h2 style={{ fontSize: '1.25rem', color: '#0f172a', margin: 0 }}>
                {queryLocation ? `Spaces near "${queryLocation}"` : 'All Available Spaces'}
                <span style={{ fontSize: '0.9rem', color: '#64748b', marginLeft: '0.5rem', fontWeight: 'normal'
            </h2>
        </div>

        {/* Loading / Results Grid */}
        {loading ? (
            <p style={{ color: '#64748b' }}>Loading available spaces...</p>
        ) : filteredListings.length === 0 ? (
            /* Empathy Empty State */
            <div style={{ backgroundColor: '#f8fafc', padding: '3rem 2rem', borderRadius: '12px', border: '1px b
                <h3 style={{ color: '#1e293b', marginBottom: '0.5rem' }}>No direct matches found in "{queryLocat
                <p style={{ color: '#64748b', maxWidth: '500px', margin: '0 auto 1.5rem auto', lineHeight: '1.5'
                    We are continuously verifying new hosts. Try searching for broader metropolitan areas, or re
                </p>
                <div style={{ display: 'flex', gap: '1rem', justifyContent: 'center' }}>
                    <button onClick={() => { setSearchInput(''); setSearchParams({}); }} style={{ padding: '0.6r
                        Clear Filters
                    </button>
                    <Link to="/register" style={{ padding: '0.6rem 1.2rem', backgroundColor: '#2563eb', color: '
                        Create Account for Help
                    </Link>
```

```
                    </div>
                </div>
            ) : (
                /* Cards Grid */
                <div style={{ display: 'grid', gridTemplateColumns: 'repeat(auto-fill, minmax(320px, 1fr))', gap: '1
                    {filteredListings.map((listing) => (
                        <div key={listing._id} style={{ backgroundColor: '#ffffff', borderRadius: '12px', border: '1
                            {listing.image ? (
                                <img src={listing.image} alt={listing.title} style={{ width: '100%', height: '180px'
                            ) : (
                                <div style={{ width: '100%', height: '140px', backgroundColor: '#f1f5f9', display: '
                                    Photo Available Upon Request
                                </div>
                            )}

                            <div style={{ padding: '1.25rem', flex: 1, display: 'flex', flexDirection: 'column', jus
                                <div>
                                    <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'fle
                                        <span style={{ backgroundColor: '#dcfce7', color: '#166534', padding: '0.2re
                                            {listing.price === 0 ? 'Free Accommodation' : `$${listing.price}/mo`}
                                        </span>
                                        <span style={{ fontSize: '0.8rem', color: '#64748b', fontWeight: '500' }}>
                                            {listing.spaceType || 'Spare Room'}
                                        </span>
                                    </div>

                                    <h3 style={{ fontSize: '1.1rem', color: '#0f172a', margin: '0.5rem 0 0.25rem 0'
                                    <p style={{ color: '#64748b', fontSize: '0.85rem', margin: 0 }}>📍 {listing.loca

                                    <div style={{ margin: '1rem 0', fontSize: '0.85rem', color: '#475569' }}>
                                        <p style={{ margin: '0.25rem 0' }}>⏱️ Stay Duration: <strong>Up to {listing.
                                        {listing.rules && <p style={{ margin: '0.25rem 0', color: '#64748b', fontSty
                                    </div>
                                </div>

                                <button
                                    onClick={handleActionClick}
                                    style={{ width: '100%', padding: '0.75rem', backgroundColor: '#2563eb', color: '
                                >
                                    {userToken ? 'Connect with Host' : 'Sign In to Request Support'}
                                </button>
                            </div>
                        </div>
                    ))}
                </div>
            )}
        </div>
    );
};

export default Search;
```

2. Update `homepage.jsx` Search Form

Connect the search bar on your homepage so it navigates to `/search?location=...`.

```
import React, { useState } from 'react';
import { useNavigate } from 'react-router-dom';

// Inside Homepage component:
const [searchQuery, setSearchQuery] = useState('');
const navigate = useNavigate();

const handleSearchSubmit = (e) => {
    e.preventDefault();
    if (searchQuery.trim()) {
        navigate(`/search?location=${encodeURIComponent(searchQuery)}`);
    } else {
        navigate('/search');
    }
};

// Inside Homepage JSX hero section:
<form onSubmit={handleSearchSubmit} style={{ display: 'flex', gap: '0.5rem', marginTop: '1.5rem' }}>
    <input
        type="text"
        placeholder="Where do you need shelter? (City or area)"
        value={searchQuery}
        onChange={(e) => setSearchQuery(e.target.value)}
```

```
        style={{ padding: '0.8rem 1rem', borderRadius: '8px', border: '1px solid #cbd5e1', width: '320px', fontSize:
    />
    <button type="submit" style={{ padding: '0.8rem 1.5rem', backgroundColor: '#2563eb', color: 'white', border: 'no
        Find Space
    </button>
</form>
```

3. Add Route in `App.jsx`

Import `Search` and define the route in your main router setup.

```
import { BrowserRouter as Router, Routes, Route } from 'react-router-dom';
import Homepage from './pages/Homepage';
import Search from './pages/Search';
import Dashboard from './pages/Dashboard';
import Login from './pages/Login';
import Register from './pages/Register';

function App() {
  return (
    <Router>
      <Routes>
        <Route path="/" element={<Homepage />} />
        <Route path="/search" element={<Search />} />
        <Route path="/dashboard" element={<Dashboard />} />
        <Route path="/login" element={<Login />} />
        <Route path="/register" element={<Register />} />
      </Routes>
    </Router>
  );
}

export default App;
```

Would you like to add additional backend filters (such as filtering by maximum duration or space type) to the search endpoint next?

User prompt: Show me where to implement the necessary changes

Response: Update **`homepage.jsx`** with the following three edits to hook up the search navigation:

1. Update Imports

At the top of the file, add `useState` to React and `useNavigate` to React Router.

```
import React, { useState, useEffect } from 'react'; // Added useState
import { useDispatch, useSelector } from 'react-redux';
import { Link, useNavigate } from 'react-router-dom'; // Added useNavigate
```

2. Add State & Navigation Handler

Inside the `Homepage` component, add a state variable for location and a form submit function right above `useEffect`:

```
const Homepage = () => {
    const dispatch = useDispatch();
    const navigate = useNavigate(); // Added navigate hook
    const { userToken } = useAuthContext();
    const { items: listings, status, error } = useSelector((state) => state.listings);

    const [location, setLocation] = useState('Tel Aviv'); // Search state

    const handleSearch = (e) => {
        e.preventDefault();
        if (location.trim()) {
            navigate(`/search?location=${encodeURIComponent(location)}`);
        } else {
            navigate('/search');
        }
    };

    useEffect(() => {
        if (status === 'idle') {
            dispatch(fetchListings());
        }
    }, [status, dispatch]);
```

3. Update the Search Container JSX

Convert `<div className="search-container">` into a `<form>` element, attach `handleSearch`, and bind the input state:

```
<form className="search-container" onSubmit={handleSearch}>
    <div className="search-group">
        <label>Location</label>
        <input
            type="text"
            placeholder="City, State, or Zip"
            value={location}
            onChange={(e) => setLocation(e.target.value)}
        />
    </div>
    <div className="search-group">
        <label>Space Type</label>
        <select defaultValue="Spare Room">
            <option>Spare Room</option>
            <option>Living Room Couch</option>
            <option>In-law Suite</option>
            <option>Entire Apartment</option>
        </select>
    </div>
    <div className="search-group">
        <label>Arrangement</label>
        <select defaultValue="Emergency (Free)">
            <option>Emergency (Free)</option>
            <option>Low-Cost Rent</option>
            <option>Work/Chore Exchange</option>
        </select>
    </div>
    <div className="search-group">
        <label>Urgency</label>
        <select defaultValue="Immediate">
            <option>Immediate / Tonight</option>
            <option>Within a week</option>
            <option>Planning ahead</option>
        </select>
    </div>
    <button type="submit" className="search-btn">
        <i className="fa-solid fa-magnifying-glass"></i> Search
    </button>
</form>
```

User prompt: This is my existing app.jsx. How to implement the suggested changes?

Response: Import `Search` at the top and add the public `/search` route inside `<Routes>`:

```
import React from 'react';
import { Routes, Route } from 'react-router-dom';
import Homepage from './components/homepage';
import Login from './components/login';
import Register from './components/register';
import Search from './components/Search'; // Import Search component
import ProtectedRoute from './components/protectedRoute';
import Dashboard from './components/dashboard';

function App() {
  return (
    <Routes>
      <Route path="/" element={<Homepage />} />
      <Route path="/search" element={<Search />} /> {/* Public Search Route */}
      <Route path="/login" element={<Login />} />
      <Route path="/register" element={<Register />} />
      <Route
        path="/dashboard"
        element={
          <ProtectedRoute>
            <Dashboard />
          </ProtectedRoute>
        }
      />
    </Routes>
  );
}

export default App;
```

User prompt: I implemented every change, and the homepage went blank again. What do I do?

Response: The blank screen occurs because `useNavigate` is called on line 10 without being imported from `'react-router-dom'` on line 3, causing an uncaught `ReferenceError` that crashes React. Additionally, the search form element on line 49 still needs to be converted from a `<div>` to a `<form>` with `onSubmit={handleSearch}` attached.

Replace the contents of your `homepage.jsx` file with this fully corrected code:

```
import React, { useState, useEffect } from 'react';
import { useDispatch, useSelector } from 'react-redux';
import { Link, useNavigate } from 'react-router-dom'; // Fixed: Added useNavigate import
import { fetchListings } from '../redux/listingsSlice';
import { useAuthContext } from '../context/AuthContext';
import './homepage.css';

const Homepage = () => {
    const dispatch = useDispatch();
    const navigate = useNavigate();
    const { userToken } = useAuthContext();
    const { items: listings, status, error } = useSelector((state) => state.listings);

    const [location, setLocation] = useState('Tel Aviv');

    const handleSearch = (e) => {
        e.preventDefault();
        if (location.trim()) {
            navigate(`/search?location=${encodeURIComponent(location)}`);
        } else {
            navigate('/search');
        }
    };

    useEffect(() => {
        if (status === 'idle') {
            dispatch(fetchListings());
        }
    }, [status, dispatch]);

    return (
        <div className="homepage">
            <header>
                <nav className="nav-container">
                    <a href="#" className="logo">
                        <i className="fa-solid fa-hands-holding-circle"></i> NestFinder
                    </a>
                    <ul className="nav-links">
                        <li><a href="#features">Our Mission</a></li>
                        <li><a href="#listings">Find Safe Spaces</a></li>
                        {userToken ? (
                            <li><Link to="/dashboard" className="btn-primary">Dashboard</Link></li>
                        ) : (
                            <>
                                <li><Link to="/login">Sign In</Link></li>
                                <li><Link to="/register" className="btn-primary">Get Started</Link></li>
                            </>
                        )}
                    </ul>
                </nav>
            </header>

            <section className="hero">
                <div className="hero-content">
                    <h1>Find a <span>Safe Place</span> to Land</h1>
                    <p>Our goal is to match you with a space where you will actually thrive, and get you out of a ne

                    {/* Fixed: Converted div to form and attached onSubmit */}
                    <form className="search-container" onSubmit={handleSearch}>
                        <div className="search-group">
                            <label>Location</label>
                            <input
                                type="text"
                                placeholder="City, State, or Zip"
                                value={location}
                                onChange={(e) => setLocation(e.target.value)}
                            />
                        </div>
                        <div className="search-group">
                            <label>Space Type</label>
                            <select defaultValue="Spare Room">
                                <option>Spare Room</option>
                                <option>Living Room Couch</option>
                                <option>In-law Suite</option>
```

```
                        <option>Entire Apartment</option>
                    </select>
                </div>
                <div className="search-group">
                    <label>Arrangement</label>
                    <select defaultValue="Emergency (Free)">
                        <option>Emergency (Free)</option>
                        <option>Low-Cost Rent</option>
                        <option>Work/Chore Exchange</option>
                    </select>
                </div>
                <div className="search-group">
                    <label>Urgency</label>
                    <select defaultValue="Immediate">
                        <option>Immediate / Tonight</option>
                        <option>Within a week</option>
                        <option>Planning ahead</option>
                    </select>
                </div>
                <button type="submit" className="search-btn">
                    <i className="fa-solid fa-magnifying-glass"></i> Search
                </button>
            </form>
        </div>
    </section>

    <section className="section" id="features">
        <div className="section-header">
            <h2>Safety and Support First</h2>
            <p>Leaving a difficult environment is hard enough. Finding a safe, supportive place to rest shou
        </div>
        <div className="features-grid">
            <div className="feature-card">
                <div className="feature-icon"><i className="fa-solid fa-shield-halved"></i></div>
                <h3>Verified Hosts</h3>
                <p>Every host offering a space goes through a community verification process and background
            </div>
            <div className="feature-card">
                <div className="feature-icon"><i className="fa-solid fa-user-lock"></i></div>
                <h3>Absolute Privacy</h3>
                <p>Your safety is our priority. Exact addresses and personal contact information are kept st
            </div>
            <div className="feature-card">
                <div className="feature-icon"><i className="fa-solid fa-handshake-angle"></i></div>
                <h3>Clear Boundaries</h3>
                <p>Our platform helps you and your host establish clear expectations, duration of stay, and
            </div>
        </div>
    </section>

    <section className="section listings-section" id="listings">
        <div className="section-header">
            <h2>Welcoming Hosts Near You</h2>
            <p>Generous community members currently offering a safe environment.</p>
        </div>

        {status === 'loading' && <p className="loading-state">Loading available spaces...</p>}
        {error && <p className="error-state">Failed to load spaces: {error}</p>}

        <div className="listings-grid">
            {listings && listings.map((listing) => (
                <div className="listing-card" key={listing._id || listing.id}>
                    <div className="listing-img" style={{ backgroundImage: `url(${listing.image || 'https://
                        <span className="tag">{listing.price === 0 || listing.price === '0' ? 'Free / Emerge
                    </div>
                    <div className="listing-content">
                        <div className="price">
                            {listing.price === 0 || listing.price === '0' ? 'No Cost' : `$${listing.price?.t
                        </div>
                        <h3 className="listing-title">{listing.title}</h3>
                        <div className="location"><i className="fa-solid fa-location-dot"></i> {listing.loca
                        <div className="details">
                            <span><i className="fa-solid fa-bed"></i> {listing.beds} Bed(s)</span>
                            <span><i className="fa-solid fa-bath"></i> {listing.baths} Bath</span>
                            <span><i className="fa-solid fa-calendar-day"></i> {listing.sqft} Days Max</span
                        </div>
                    </div>
                </div>
            ))}
```

```
                </div>
            </section>

            <footer>
                <div className="footer-container">
                    <div className="footer-info">
                        <a href="#" className="logo">
                            <i className="fa-solid fa-hands-holding-circle"></i> NestFinder
                        </a>
                        <p>Helping you transition safely into a space where you can heal and rebuild.</p>
                    </div>
                </div>
                <div className="copyright">
                    &copy; 2026 NestFinder Mutual Aid. All rights reserved.
                </div>
            </footer>
        </div>
    );
};

export default Homepage;
```

User prompt: Searching while logged in, I get this. Let's change that

Response: To fix this, update the empty state section in **`Search.jsx`** to dynamically check for `userToken`. When logged in, it will show a relevant message and link to the **Dashboard** instead of asking the user to register.

Replace the empty state block in **`Search.jsx`** with this dynamic condition:

```
{/* Empathy Empty State */}
<div style={{ backgroundColor: '#f8fafc', padding: '3rem 2rem', borderRadius: '12px', border: '1px solid #e2e8f0', t
    <h3 style={{ color: '#1e293b', marginBottom: '0.5rem' }}>
        No direct matches found in "{queryLocation}"
    </h3>

    <p style={{ color: '#64748b', maxWidth: '500px', margin: '0 auto 1.5rem auto', lineHeight: '1.5', fontSize: '0.9
        {userToken ? (
            "We are continuously verifying new hosts. Try searching for broader metropolitan areas, or head to your
        ) : (
            "We are continuously verifying new hosts. Try searching for broader metropolitan areas, or register to s
        )}
    </p>

    <div style={{ display: 'flex', gap: '1rem', justifyContent: 'center' }}>
        <button
            onClick={() => { setSearchInput(''); setSearchParams({}); }}
            style={{ padding: '0.6rem 1.2rem', backgroundColor: '#e2e8f0', color: '#334155', border: 'none', borderR
        >
            Clear Filters
        </button>

        {userToken ? (
            <Link
                to="/dashboard"
                style={{ padding: '0.6rem 1.2rem', backgroundColor: '#2563eb', color: '#ffffff', borderRadius: '6px'
            >
                Go to Dashboard
            </Link>
        ) : (
            <Link
                to="/register"
                style={{ padding: '0.6rem 1.2rem', backgroundColor: '#2563eb', color: '#ffffff', borderRadius: '6px'
            >
                Create Account for Help
            </Link>
        )}
    </div>
</div>
```

User prompt: After implementing your changes, the page is full of bugs. Help me fix

Response: The main crash is caused by a **JSX syntax error on line 84**. The comment `{/* Empathy Empty State */}` was placed inside parentheses `(` directly before a JSX tag, which causes the React compiler to fail.

Additionally, `searchInput` needed a `useEffect` sync so the search input box updates properly when clearing filters or navigating with URL parameters.

Here is the complete, bug-free **search.jsx** file:

```
import React, { useState, useEffect } from 'react';
import axios from 'axios';
import { useSearchParams, useNavigate, Link } from 'react-router-dom';
import { useAuthContext } from '../context/AuthContext';

const Search = () => {
    const [searchParams, setSearchParams] = useSearchParams();
    const queryLocation = searchParams.get('location') || '';

    const [searchInput, setSearchInput] = useState(queryLocation);
    const [listings, setListings] = useState([]);
    const [filteredListings, setFilteredListings] = useState([]);
    const [loading, setLoading] = useState(true);

    const { userToken } = useAuthContext();
    const navigate = useNavigate();

    useEffect(() => {
        fetchListings();
    }, []);

    // Sync input field when URL parameter changes
    useEffect(() => {
        setSearchInput(queryLocation);
    }, [queryLocation]);

    // Filter listings whenever queryLocation or raw listings change
    useEffect(() => {
        if (queryLocation.trim() === '') {
            setFilteredListings(listings);
        } else {
            const filtered = listings.filter(item =>
                item.location?.toLowerCase().includes(queryLocation.toLowerCase()) ||
                item.title?.toLowerCase().includes(queryLocation.toLowerCase())
            );
            setFilteredListings(filtered);
        }
    }, [queryLocation, listings]);

    const fetchListings = async () => {
        try {
            const res = await axios.get('http://localhost:5000/api/listings');
            setListings(res.data);
        } catch (err) {
            console.error('Failed to load listings', err);
        } finally {
            setLoading(false);
        }
    };

    const handleSearchSubmit = (e) => {
        e.preventDefault();
        setSearchParams({ location: searchInput });
    };

    const handleActionClick = () => {
        if (!userToken) {
            navigate('/register');
        } else {
            navigate('/dashboard');
        }
    };

    return (
        <div style={{ maxWidth: '1100px', margin: '2rem auto', padding: '0 1.5rem' }}>
            {/* Header & Search Bar */}
            <div style={{ backgroundColor: '#ffffff', padding: '2rem', borderRadius: '12px', border: '1px solid #e2e
                <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', marginBottom:
                    <Link to="/" style={{ textDecoration: 'none', color: '#2563eb', fontWeight: 'bold', fontSize: '1
                        ← Back to Home
                    </Link>
                    {!userToken && (
                        <div style={{ fontSize: '0.9rem', color: '#64748b' }}>
                            Need to host or request support? <Link to="/login" style={{ color: '#2563eb', fontWeight
                        </div>
                    )}
                </div>
```

```
    <h1 style={{ fontSize: '1.75rem', color: '#0f172a', marginBottom: '0.5rem' }}>Find Safe Accommodatio
    <p style={{ color: '#64748b', marginBottom: '1.5rem', fontSize: '0.95rem' }}>Search vetted host spac

    <form onSubmit={handleSearchSubmit} style={{ display: 'flex', gap: '0.75rem', maxWidth: '600px' }}>
        <input
            type="text"
            placeholder="Enter city or neighborhood (e.g. Tel Aviv)..."
            value={searchInput}
            onChange={(e) => setSearchInput(e.target.value)}
            style={{ flex: 1, padding: '0.75rem 1rem', borderRadius: '8px', border: '1px solid #cbd5e1',
        />
        <button type="submit" style={{ backgroundColor: '#2563eb', color: '#ffffff', border: 'none', pad
            Search
        </button>
    </form>
</div>

{/* Results Header */}
<div style={{ marginBottom: '1.5rem' }}>
    <h2 style={{ fontSize: '1.25rem', color: '#0f172a', margin: 0 }}>
        {queryLocation ? `Spaces near "${queryLocation}"` : 'All Available Spaces'}
        <span style={{ fontSize: '0.9rem', color: '#64748b', marginLeft: '0.5rem', fontWeight: 'normal'
    </h2>
</div>

{/* Loading / Results Grid */}
{loading ? (
    <p style={{ color: '#64748b' }}>Loading available spaces...</p>
) : filteredListings.length === 0 ? (
    /* Empathy Empty State */
    <div style={{ backgroundColor: '#f8fafc', padding: '3rem 2rem', borderRadius: '12px', border: '1px s
        <h3 style={{ color: '#1e293b', marginBottom: '0.5rem' }}>
            No direct matches found in "{queryLocation}"
        </h3>

        <p style={{ color: '#64748b', maxWidth: '500px', margin: '0 auto 1.5rem auto', lineHeight: '1.5'
            {userToken ? (
                "We are continuously verifying new hosts. Try searching for broader metropolitan areas,
            ) : (
                "We are continuously verifying new hosts. Try searching for broader metropolitan areas,
            )}
        </p>

        <div style={{ display: 'flex', gap: '1rem', justifyContent: 'center' }}>
            <button
                onClick={() => { setSearchInput(''); setSearchParams({}); }}
                style={{ padding: '0.6rem 1.2rem', backgroundColor: '#e2e8f0', color: '#334155', border:
            >
                Clear Filters
            </button>

            {userToken ? (
                <Link
                    to="/dashboard"
                    style={{ padding: '0.6rem 1.2rem', backgroundColor: '#2563eb', color: '#ffffff', bor
                >
                    Go to Dashboard
                </Link>
            ) : (
                <Link
                    to="/register"
                    style={{ padding: '0.6rem 1.2rem', backgroundColor: '#2563eb', color: '#ffffff', bor
                >
                    Create Account for Help
                </Link>
            )}
        </div>
    </div>
) : (
    /* Cards Grid */
    <div style={{ display: 'grid', gridTemplateColumns: 'repeat(auto-fill, minmax(320px, 1fr))', gap: '1
        {filteredListings.map((listing) => (
            <div key={listing._id || listing.id} style={{ backgroundColor: '#ffffff', borderRadius: '12p
                {listing.image ? (
                    <img src={listing.image} alt={listing.title} style={{ width: '100%', height: '180px'
                ) : (
                    <div style={{ width: '100%', height: '140px', backgroundColor: '#f1f5f9', display: '
                        Photo Available Upon Request
```

```
                            </div>
                        )}

                        <div style={{ padding: '1.25rem', flex: 1, display: 'flex', flexDirection: 'column', jus
                            <div>
                                <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'fle
                                    <span style={{ backgroundColor: '#dcfce7', color: '#166534', padding: '0.2re
                                        {listing.price === 0 || listing.price === '0' ? 'Free Accommodation' : `
                                    </span>
                                    <span style={{ fontSize: '0.8rem', color: '#64748b', fontWeight: '500' }}>
                                        {listing.spaceType || 'Spare Room'}
                                    </span>
                                </div>

                                <h3 style={{ fontSize: '1.1rem', color: '#0f172a', margin: '0.5rem 0 0.25rem 0'
                                <p style={{ color: '#64748b', fontSize: '0.85rem', margin: 0 }}>📍 {listing.loca

                                <div style={{ margin: '1rem 0', fontSize: '0.85rem', color: '#475569' }}>
                                    <p style={{ margin: '0.25rem 0' }}>⏱️ Stay Duration: <strong>Up to {listing.
                                    {listing.rules && <p style={{ margin: '0.25rem 0', color: '#64748b', fontSty
                                </div>
                            </div>

                            <button
                                onClick={handleActionClick}
                                style={{ width: '100%', padding: '0.75rem', backgroundColor: '#2563eb', color: '
                            >
                                {userToken ? 'Connect with Host' : 'Sign In to Request Support'}
                            </button>
                        </div>
                    </div>
                ))}
            </div>
        )}
    </div>
    );
};

export default Search;
```

User prompt: As host, I get no update about seekers who contacted me. And as a seeker, the "request support" button does nothing. Let's fix that

Response: To fix this, we need to create a **Request system**: a database model for support inquiries, backend endpoints to post and retrieve requests, and UI updates for both the seeker (to send requests) and host (to view incoming requests).

Step 1: Create Request Model & Route (Backend)

Create **`backend/models/Request.js`**:

```
const mongoose = require('mongoose');

const RequestSchema = new mongoose.Schema({
    listing: { type: mongoose.Schema.Types.ObjectId, ref: 'Listing', required: true },
    seeker: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
    host: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
    message: { type: String, default: 'Interested in this space for support.' },
    status: { type: String, enum: ['pending', 'accepted', 'declined'], default: 'pending' },
    createdAt: { type: Date, default: Date.now }
});

module.exports = mongoose.model('Request', RequestSchema);
```

Create **`backend/routes/requestRoutes.js`**:

```
const express = require('express');
const router = express.Router();
const Request = require('../models/Request');
const Listing = require('../models/Listing');

// POST /api/requests - Seeker sends request
router.post('/', async (req, res) => {
    try {
        const { listingId, message, seekerId } = req.body;
        const listing = await Listing.findById(listingId);
        if (!listing) return res.status(404).json({ message: 'Listing not found' });

        const newRequest = new Request({
```

```
            listing: listingId,
            seeker: seekerId,
            host: listing.user || listing.host,
            message: message || 'I need support and would like to connect.'
        });

        await newRequest.save();
        res.status(201).json({ message: 'Support request sent successfully!', request: newRequest });
    } catch (err) {
        res.status(500).json({ message: err.message });
    }
});

// GET /api/requests/host/:hostId - Host views incoming requests
router.get('/host/:hostId', async (req, res) => {
    try {
        const requests = await Request.find({ host: req.params.hostId })
            .populate('seeker', 'email role')
            .populate('listing', 'title location');
        res.json(requests);
    } catch (err) {
        res.status(500).json({ message: err.message });
    }
});

module.exports = router;
```

Register the route in **`server.js`**:

```
const requestRoutes = require('./routes/requestRoutes');
// ... after other app.use routes
app.use('/api/requests', requestRoutes);
```

Step 2: Hook Up "Request Support" Button (`Search.jsx`)

Update the `handleActionClick` handler in **`Search.jsx`** so clicking the button actually creates a request in the database:

```
const handleActionClick = async (listing) => {
    if (!userToken) {
        navigate('/login');
        return;
    }

    try {
        await axios.post('http://localhost:5000/api/requests', {
            listingId: listing._id,
            seekerId: user?.id || user?._id, // Ensure user ID comes from AuthContext
            message: "Hello, I am requesting support for this safe space."
        });
        alert(`Support request successfully sent to the host for "${listing.title}"!`);
    } catch (err) {
        alert(err.response?.data?.message || 'Failed to send request');
    }
};
```

Update line in **`Search.jsx`** where the button is rendered:

```
<button
    onClick={() => handleActionClick(listing)}
    style={{ width: '100%', padding: '0.75rem', backgroundColor: '#2563eb', color: '#ffffff', border: 'none', border
>
    {userToken ? 'Request Support' : 'Sign In to Request Support'}
</button>
```

Step 3: Add Host Request Inbox (`Dashboard.jsx`)

Add an incoming inquiries widget inside **`Dashboard.jsx`** so hosts can view seekers who reached out to them:

```
import React, { useState, useEffect } from 'react';
import axios from 'axios';
import { useAuthContext } from '../context/AuthContext';

const HostRequests = () => {
    const { user } = useAuthContext();
    const [requests, setRequests] = useState([]);
    const [loading, setLoading] = useState(true);

    useEffect(() => {
        if (user?.id || user?._id) {
            axios.get(`http://localhost:5000/api/requests/host/${user.id || user._id}`)
```

```
                .then(res => setRequests(res.data))
                .catch(err => console.error(err))
                .finally(() => setLoading(false));
        }
    }, [user]);

    if (loading) return <p>Loading inquiries...</p>;

    return (
        <div style={{ marginTop: '2rem', backgroundColor: '#ffffff', padding: '1.5rem', borderRadius: '12px', border
            <h3 style={{ margin: '0 0 1rem 0', color: '#0f172a' }}>📬 Incoming Support Requests ({requests.length})<
            {requests.length === 0 ? (
                <p style={{ color: '#64748b' }}>No seekers have contacted you yet.</p>
            ) : (
                <div style={{ display: 'flex', flexDirection: 'column', gap: '1rem' }}>
                    {requests.map((req) => (
                        <div key={req._id} style={{ border: '1px solid #cbd5e1', padding: '1rem', borderRadius: '8px
                            <p style={{ margin: '0 0 0.25rem 0', fontWeight: '600', color: '#1e293b' }}>
                                Listing: {req.listing?.title || 'Safe Space'} ({req.listing?.location})
                            </p>
                            <p style={{ margin: '0 0 0.5rem 0', fontSize: '0.9rem', color: '#475569' }}>
                                👤 Seeker Contact: <strong>{req.seeker?.email}</strong>
                            </p>
                            <p style={{ margin: 0, fontStyle: 'italic', color: '#64748b', fontSize: '0.85rem' }}>
                                "{req.message}"
                            </p>
                        </div>
                    ))}
                </div>
            )}
        </div>
    );
};

export default HostRequests;
```

User prompt: rewrite the dashboard to accommodate the newest changes

Response: Here is the updated **`Dashboard.jsx`** file with full support for incoming host inquiries and active seeker support requests:

```
import React, { useState, useEffect } from 'react';
import axios from 'axios';
import { useAuthContext } from '../context/AuthContext';
import { useNavigate } from 'react-router-dom';
import './homepage.css';

const Dashboard = () => {
    const { user, userToken, logoutUser } = useAuthContext();
    const navigate = useNavigate();

    // Host Form State
    const [formData, setFormData] = useState({
        title: '',
        location: '',
        spaceType: 'Spare Room',
        price: 0,
        beds: 1,
        baths: 1,
        maxDuration: 14,
        rules: 'Quiet hours after 10 PM, pet friendly',
        image: ''
    });

    const [myListings, setMyListings] = useState([]);
    const [requests, setRequests] = useState([]);
    const [loading, setLoading] = useState(false);
    const [message, setMessage] = useState('');

    const userId = user?.id || user?._id;

    // Fetch listings and requests
    useEffect(() => {
        fetchListings();
        if (userId) {
            fetchRequests();
        }
    }, [userId]);
```

```
const fetchListings = async () => {
    try {
        const res = await axios.get('http://localhost:5000/api/listings');
        if (user?.role === 'host' && userId) {
            setMyListings(res.data.filter(item => item.user === userId || item.user?._id === userId));
        } else {
            setMyListings(res.data);
        }
    } catch (err) {
        console.error('Failed to fetch listings', err);
    }
};

const fetchRequests = async () => {
    try {
        if (user?.role === 'host') {
            const res = await axios.get(`http://localhost:5000/api/requests/host/${userId}`);
            setRequests(res.data);
        }
    } catch (err) {
        console.error('Failed to fetch requests', err);
    }
};

const handleFormSubmit = async (e) => {
    e.preventDefault();
    setLoading(true);
    setMessage('');

    try {
        await axios.post(
            'http://localhost:5000/api/listings',
            formData,
            { headers: { Authorization: `Bearer ${userToken}` } }
        );
        setMessage('Space posted successfully!');

        setFormData({
            title: '',
            location: '',
            spaceType: 'Spare Room',
            price: 0,
            beds: 1,
            baths: 1,
            maxDuration: 14,
            rules: '',
            image: ''
        });

        fetchListings();
    } catch (err) {
        setMessage(err.response?.data?.message || 'Failed to post space');
    } finally {
        setLoading(false);
    }
};

const handleDelete = async (id) => {
    if (!window.confirm('Are you sure you want to remove this space?')) return;
    try {
        await axios.delete(`http://localhost:5000/api/listings/${id}`, {
            headers: { Authorization: `Bearer ${userToken}` }
        });
        fetchListings();
    } catch (err) {
        alert('Failed to delete listing');
    }
};

const handleRequestSupport = async (listing) => {
    try {
        await axios.post('http://localhost:5000/api/requests', {
            listingId: listing._id,
            seekerId: userId,
            message: "Hello, I am requesting support for this safe space."
        });
        alert(`Support request sent successfully to the host for "${listing.title}"!`);
    } catch (err) {
```

```
            alert(err.response?.data?.message || 'Failed to send support request');
        }
    };

    const handleImageUpload = (e) => {
        const file = e.target.files[0];
        if (file) {
            if (file.size > 2 * 1024 * 1024) {
                alert('File size is too large. Please select an image under 2MB.');
                return;
            }
            const reader = new FileReader();
            reader.onloadend = () => {
                setFormData(prev => ({ ...prev, image: reader.result }));
            };
            reader.readAsDataURL(file);
        }
    };

    const quickExit = () => {
        window.location.href = 'https://www.google.com';
    };

    return (
        <div style={{ maxWidth: '1100px', margin: '2rem auto', padding: '0 1.5rem' }}>
            {/* Top Navigation Bar */}
            <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', paddingBottom: '1.
                <div>
                    <h1 style={{ fontSize: '1.75rem', color: '#0f172a', margin: 0 }}>
                        {user?.role === 'host' ? 'Host Community Portal' : 'Seeker Support Portal'}
                    </h1>
                    <p style={{ color: '#64748b', margin: '0.25rem 0 0 0', fontSize: '0.95rem' }}>
                        Logged in as <strong>{user?.email}</strong> ({user?.role?.toUpperCase() || 'MEMBER'})
                    </p>
                </div>
                <div style={{ display: 'flex', gap: '1rem', alignItems: 'center' }}>
                    {user?.role === 'seeker' && (
                        <button
                            onClick={quickExit}
                            style={{ backgroundColor: '#dc2626', color: '#fff', border: 'none', padding: '0.6rem 1re
                            <i className="fa-solid fa-person-running"></i> Quick Exit
                        </button>
                    )}
                    <button
                        onClick={() => { logoutUser(); navigate('/'); }}
                        style={{ backgroundColor: 'transparent', border: '1px solid #cbd5e1', padding: '0.6rem 1rem'
                        Sign Out
                    </button>
                </div>
            </div>

            {/* HOST DASHBOARD VIEW */}
            {user?.role === 'host' ? (
                <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '2rem', marginTop: '2rem' }}>
                    {/* Post Space Form */}
                    <div style={{ backgroundColor: '#ffffff', padding: '1.75rem', borderRadius: '12px', border: '1px
                        <h2 style={{ fontSize: '1.25rem', marginBottom: '1rem', color: '#0f172a' }}>Offer a Safe Spa
                        {message && <p style={{ padding: '0.5rem', borderRadius: '4px', backgroundColor: message.inc

                        <form onSubmit={handleFormSubmit} style={{ display: 'flex', flexDirection: 'column', gap: '1
                            <div>
                                <label style={{ display: 'block', fontSize: '0.85rem', fontWeight: '600', marginBott
                                <input type="text" placeholder="e.g. Peaceful Private Bedroom" value={formData.title
                            </div>
                            <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '1rem' }}>
                                <div>
                                    <label style={{ display: 'block', fontSize: '0.85rem', fontWeight: '600', margin
                                    <input type="text" placeholder="e.g. Tel Aviv" value={formData.location} onChang
                                </div>
                                <div>
                                    <label style={{ display: 'block', fontSize: '0.85rem', fontWeight: '600', margin
                                    <select value={formData.spaceType} onChange={(e) => setFormData({...formData, sp
                                        <option value="Spare Room">Spare Room</option>
                                        <option value="Living Room Couch">Living Room Couch</option>
                                        <option value="In-law Suite">In-law Suite</option>
                                        <option value="Entire Apartment">Entire Apartment</option>
                                    </select>
                                </div>
                            </div>
```

```
                <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr 1fr', gap: '0.75rem' }}>
                    <div>
                        <label style={{ display: 'block', fontSize: '0.85rem', fontWeight: '600', margin
                        <input type="number" min="0" value={formData.price} onChange={(e) => setFormData
                    </div>
                    <div>
                        <label style={{ display: 'block', fontSize: '0.85rem', fontWeight: '600', margin
                        <input type="number" min="1" value={formData.maxDuration} onChange={(e) => setFo
                    </div>
                    <div>
                        <label style={{ display: 'block', fontSize: '0.85rem', fontWeight: '600', margin
                        <input type="number" min="1" value={formData.beds} onChange={(e) => setFormData(
                    </div>
                </div>
                <div>
                    <label style={{ display: 'block', fontSize: '0.85rem', fontWeight: '600', marginBott
                    <textarea rows="3" value={formData.rules} onChange={(e) => setFormData({...formData,
                </div>
                <div>
                    <label style={{ display: 'block', fontSize: '0.85rem', fontWeight: '600', marginBott
                    <input
                        type="file"
                        accept="image/*"
                        onChange={handleImageUpload}
                        style={{ width: '100%', padding: '0.5rem', borderRadius: '6px', border: '1px sol
                    />
                    {formData.image && (
                        <div style={{ marginTop: '0.5rem' }}>
                            <img
                                src={formData.image}
                                alt="Preview"
                                style={{ width: '100%', height: '140px', objectFit: 'cover', borderRadiu
                            />
                        </div>
                    )}
                </div>
                <button type="submit" disabled={loading} style={{ padding: '0.75rem', backgroundColor: '
                    {loading ? 'Publishing...' : 'Publish Space Offer'}
                </button>
            </form>
        </div>

        {/* Right Panel: Incoming Requests & Offered Spaces */}
        <div style={{ display: 'flex', flexDirection: 'column', gap: '2rem' }}>
            {/* Incoming Support Requests */}
            <div style={{ backgroundColor: '#ffffff', padding: '1.5rem', borderRadius: '12px', border: '
                <h2 style={{ fontSize: '1.25rem', marginBottom: '1rem', color: '#0f172a' }}>📬 Incoming
                {requests.length === 0 ? (
                    <p style={{ color: '#64748b', fontSize: '0.9rem' }}>No seekers have contacted you ye
                ) : (
                    <div style={{ display: 'flex', flexDirection: 'column', gap: '1rem' }}>
                        {requests.map((req) => (
                            <div key={req._id} style={{ border: '1px solid #cbd5e1', padding: '1rem', bo
                                <p style={{ margin: '0 0 0.25rem 0', fontWeight: '600', color: '#1e293b'
                                    {req.listing?.title || 'Safe Space'} ({req.listing?.location})
                                </p>
                                <p style={{ margin: '0 0 0.5rem 0', fontSize: '0.85rem', color: '#475569
                                    👤 Seeker Contact: <strong>{req.seeker?.email}</strong>
                                </p>
                                <p style={{ margin: 0, fontStyle: 'italic', color: '#64748b', fontSize:
                                    "{req.message}"
                                </p>
                            </div>
                        ))}
                    </div>
                )}
            </div>

            {/* Manage Posted Spaces */}
            <div>
                <h2 style={{ fontSize: '1.25rem', marginBottom: '1rem', color: '#0f172a' }}>Your Offered
                {myListings.length === 0 ? (
                    <p style={{ color: '#64748b' }}>You haven't posted any spaces yet.</p>
                ) : (
                    <div style={{ display: 'flex', flexDirection: 'column', gap: '1rem' }}>
                        {myListings.map((item) => (
                            <div key={item._id} style={{ backgroundColor: '#ffffff', padding: '1.25rem',
                                <div>
                                    <h3 style={{ margin: 0, fontSize: '1rem', color: '#0f172a' }}>{item.
```

```
                                        <p style={{ margin: '0.25rem 0', color: '#64748b', fontSize: '0.85re
                                        <span style={{ fontSize: '0.75rem', backgroundColor: '#f1f5f9', padd
                                    </div>
                                    <button onClick={() => handleDelete(item._id)} style={{ backgroundColor:
                                        Remove
                                    </button>
                                </div>
                            ))}
                        </div>
                    )}
                </div>
            </div>
        </div>
    ) : (
        /* SEEKER DASHBOARD VIEW */
        <div style={{ marginTop: '2rem' }}>
            <div style={{ backgroundColor: '#f0f9ff', border: '1px solid #bae6fd', padding: '1.5rem', border
                <h2 style={{ color: '#0369a1', marginTop: 0, fontSize: '1.25rem' }}>Safety & Privacy Checkli
                <ul style={{ color: '#0c4a6e', paddingLeft: '1.25rem', margin: '0.5rem 0 0 0', lineHeight: '
                    <li>Never share your exact current location until you feel comfortable with a verified h
                    <li>Use our internal messaging to communicate before meeting.</li>
                    <li>Use the red <strong>Quick Exit</strong> button at the top right to immediately leave
                </ul>
            </div>

            <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', marginBott
                <h2 style={{ fontSize: '1.25rem', color: '#0f172a', margin: 0 }}>Explore Safe Spaces Nearby<
                <button onClick={() => navigate('/search')} style={{ backgroundColor: '#2563eb', color: '#ff
                    Browse All Postings
                </button>
            </div>

            <div style={{ display: 'grid', gridTemplateColumns: 'repeat(auto-fill, minmax(300px, 1fr))', gap
                {myListings.slice(0, 6).map((listing) => (
                    <div key={listing._id} style={{ backgroundColor: '#ffffff', borderRadius: '10px', paddin
                        <div>
                            <span style={{ backgroundColor: '#dcfce7', color: '#166534', padding: '0.2rem 0.
                                {listing.price === 0 ? 'Free Accommodation' : `$${listing.price}/mo`}
                            </span>
                            <h3 style={{ fontSize: '1.1rem', margin: '0.75rem 0 0.25rem 0' }}>{listing.title
                            <p style={{ color: '#64748b', fontSize: '0.85rem', margin: 0 }}>📍 {listing.loca
                            <div style={{ margin: '1rem 0', fontSize: '0.85rem', color: '#334155' }}>
                                <p style={{ margin: '0.25rem 0' }}>⏱️ Max Stay: <strong>{listing.maxDuratior
                                <p style={{ margin: '0.25rem 0' }}>🏠 Space: <strong>{listing.spaceType || '
                            </div>
                        </div>
                        <button
                            onClick={() => handleRequestSupport(listing)}
                            style={{ width: '100%', padding: '0.6rem', backgroundColor: '#2563eb', color: '#
                        >
                            Request Support
                        </button>
                    </div>
                ))}
            </div>
        </div>
    )}
  </div>
  );
};

export default Dashboard;
```

User prompt: Thank you

Response: You're very welcome! Glad we got the support request flow and dashboard updates wired up smoothly.

Let me know if you run into any edge cases with the backend endpoints or UI components!
